

JV32 System-on-Chip Datasheet

JV32 Development Team

Version 1.4, 2026-09-15

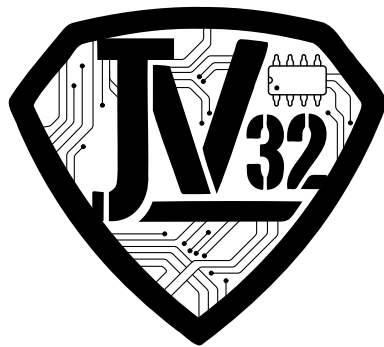


Table of Contents

1. Introduction	2
1.1. SoC Overview	2
1.2. Document Conventions	2
1.3. Scope and Implementation Notes	3
1.4. Compliance Status and Verification Boundary	3
1.5. Default Configuration	4
1.6. Key Features	5
2. System Architecture	6
2.1. Block Diagram	6
2.2. System Components	6
2.2.1. Processor and Memory Integration	6
2.2.2. TCM: IRAM and DRAM (<code>jv32_top</code>)	7
2.2.3. Merged AXI4-Lite Master	7
2.2.4. AXI4-Lite Slave	8
2.2.5. Peripheral AXI Crossbar	8
2.3. Register Table Conventions	9
3. Memory Map	10
3.1. Address Space Overview	10
3.2. Address Decode Rules	11
3.2.1. TCM Decode (inside <code>jv32_top</code>)	11
3.2.2. Peripheral Crossbar Decode	11
4. Processor Core Specification	13
4.1. Data Representation	13
4.2. ISA Support	13
4.3. RV32E Configuration	13
4.4. Pipeline Architecture	14
4.5. Forwarding and Hazards	15
4.6. Atomic Operation Semantics	16
4.6.1. LR/SC Reservation	16
4.6.2. AMO Encoding Scope	17
4.6.3. Atomic Alignment	17
4.7. Control-Flow Prediction	17
4.7.1. Conditional Branch Prediction	17
4.7.2. JAL	18
4.7.3. JALR	18

4.7.4. Return Address Stack	18
4.8. RTL Operation Timing	19
4.9. CSR (Control and Status Registers)	19
4.10. Exception and Interrupt Handling	22
4.10.1. Exception Types	22
4.10.2. Exception Detection Priority	23
4.10.3. Redirect and Trap Priority	23
4.10.4. Interrupt Types	24
4.10.5. Trap Handling Sequence	24
4.10.6. Tail-Chaining (Hardware)	24
4.10.7. Software-Assisted Tail-Chain (mnxti)	25
5. TCM Specification	26
5.1. Overview	26
5.2. Access Priorities	26
5.3. TCM Timing	27
5.4. TCM AXI Slave Semantics	27
5.5. ELF Loading (Pre-Simulation)	28
6. Peripheral Specifications	29
6.1. UART (Universal Asynchronous Receiver/Transmitter)	29
6.1.1. UART Overview	29
6.1.2. UART Register Map	29
6.1.3. Baud Rate Configuration	31
6.1.4. Interrupt Behavior	32
6.1.5. External Pins	32
6.2. Local Interrupt Controller (axi_clic)	32
6.2.1. Overview	32
6.2.2. Register Map	33
6.2.3. Timer Operation	34
6.2.4. Software Interrupt Operation	35
6.2.5. External Interrupt Operation	35
6.3. Magic Device (Simulation Only)	36
6.3.1. Magic Device Overview	36
6.3.2. Magic Device Register Map	36
6.3.3. AXI Response Behavior	36
6.3.4. Usage Example	37
7. AXI Interconnect and Bus Protocol	38
7.1. Protocol Scope	38

7.2. AXI4-Lite Overview	38
7.3. <code>jv32_top</code> AXI Master Port (<code>m_axi_*</code>)	38
7.4. <code>jv32_top</code> AXI Slave Port (<code>s_axi_*</code>)	39
7.5. Crossbar Architecture	39
7.6. Bus State Machines (inside <code>jv32_top</code>)	40
8. Debug and JTAG	41
8.1. Debug Architecture	41
8.2. JTAG TAP	41
8.3. DTMCS	42
8.4. DMI Register Map	43
8.5. Abstract Commands	45
8.5.1. Access Register	45
8.5.2. Access Memory	45
8.6. System Bus Access (SBA)	45
8.7. Debug CSRs	46
8.8. Hardware Triggers	46
8.9. cjTAG Bridge	48
9. Clock, Reset, and RTL Timing	49
9.1. Clock and Reset	49
9.1.1. Clock Domains	49
9.1.2. System Clock (<code>clk</code>)	49
9.1.3. TAP Clock — <code>USE_CJTAG=0</code> (4-wire JTAG)	50
9.1.4. TAP Clock — <code>USE_CJTAG=1</code> (2-wire cjTAG)	50
9.1.5. Reset Architecture	50
9.2. Timing Parameters	53
9.3. Top-Level Parameters	53
10. Programming Guide	56
10.1. MMIO Access Rules	56
10.2. Access and Error Handling	56
10.3. Boot Sequence	56
10.4. Stack Configuration	57
10.5. Interrupt Handling	57
10.5.1. Setting Up Interrupts	57
10.5.2. Trap Handler	57
10.6. UART Usage	58
10.7. Simulation Debugging	59
10.7.1. Magic Device Console	59

10.7.2. Waveform Capture	59
10.7.3. Instruction Trace	60
10.8. Performance-Oriented Programming Considerations	60
11. Register Reference Index	61
Appendix A: Trap Cause Reference	62
Appendix B: Pin Descriptions	63
B.1. Clock and Reset Pins	63
B.2. UART Interface	63
B.3. External Interrupt Interface	63
B.4. TCM AXI Slave Interface	63
B.5. JTAG / cJTAG Debug Interface	64
B.6. Trace Interface	65
B.7. Branch Predictor Observability (perf_bp_*)	67
B.8. Heartbeat Output	67
B.9. External AXI Master Interface (ext_axi_*)	68
12. Release and Integration Checklist	70
13. Verification and Compliance Status	71
14. Revision History	73
15. References	74
15.1. Verification Boundary	74

JV32 is a compact 32-bit RISC-V System-on-Chip (SoC) for RTL simulation and FPGA prototyping. It integrates a single-issue 3-stage processor core, instruction and data TCM, AXI4-Lite interfaces, UART, a project-specific local interrupt controller with CLINT-style timer/software-interrupt registers, simulation support, and JTAG/cJTAG debug connectivity.

This datasheet describes the implemented RTL behavior. Unless stated otherwise, addresses, reset values, timing, and feature descriptions refer to the default RTL parameters.

Chapter 1. Introduction

1.1. SoC Overview

The JV32 SoC is a compact embedded processor system. The default configuration implements RV32IMAC with M-mode (M-mode). The main blocks are:

- **Processor core** (`juv32_core`): single-issue 3-stage pipeline (IF → EX → WB), RVC expansion, configurable multiply/divide/shift units, branch prediction, and atomic-operation support
- **Tightly Coupled Memories (TCM)**: 128 KB IRAM at `0x8000_0000` and 128 KB DRAM at `0x9000_0000` by default, with 1-cycle read latency
- **Merged AXI4-Lite Master**: Out-of-TCM accesses from both I-fetch and D-bus share one master port; D-bus has priority over I-fetch
- **AXI4-Lite Slave**: External agents (debug master, testbench DPI) can read/write TCM; core has absolute priority
- **Peripheral Subsystem** (4-target AXI crossbar): UART, local interrupt controller, Magic device, external catch-all
- **Interrupt System**: local interrupt controller with CLINT-style 64-bit timer (`mtime/mtimecmp`), software interrupt (`msip`), and 16 prioritized external interrupt lines. The register model is implementation-specific and must not be described as full RISC-V CLIC compliance.

1.2. Document Conventions

This datasheet describes the behavior implemented by the supplied RTL. Unless a section explicitly states otherwise:

- Address ranges and reset values refer to the default RTL parameters unless explicitly marked as parameter-dependent.
- `R/O`, `R/W`, and `W/O` mean read-only, read/write, and write-only.
- Multi-byte values use little-endian byte order.
- AXI response names use the AXI4-Lite encodings `OKAY`, `SLVERR`, and `DECERR`.
- Signal names and RTL parameter names are shown in monospace, for example `ext_irq_i` and `IRAM_SIZE`.
- Optional features are identified by the RTL parameter that enables them.
- Software must not rely on undocumented or reserved register bits. Unless explicitly stated otherwise, software should write zero to reserved bits and ignore reserved bits when reading.

- “Implemented” means that an active RTL path provides the stated behavior; a declared constant or unused encoding alone is not treated as an implemented feature.
- **must** identifies a software or integration requirement. **should** identifies a recommended practice. Descriptive statements define RTL behavior.
- Reset values apply to the corresponding RTL state after reset has been asserted. Software-visible state may change immediately after reset is released as counters, timers, or execution begin.

1.3. Scope and Implementation Notes

The datasheet is an RTL functional specification. It does not define FPGA- or ASIC-specific timing closure, maximum clock frequency, power, area, or electrical characteristics. Those values depend on the target technology, synthesis constraints, and physical implementation.

Unless stated otherwise, sizes, counts, base addresses, and enabled features use the RTL default parameters. If an integration changes a parameter, the corresponding feature, memory size, or ISA capability may also change.

Behavior outside the documented address ranges, register encodings, access sizes, or enabled feature set is not part of the software interface unless the relevant section explicitly defines it.

Microarchitectural details such as pipeline timing, arbitration timing, and internal state transitions describe the current RTL implementation. Software must not depend on cycle-exact behavior unless a section explicitly defines that behavior as part of the interface.

1.4. Compliance Status and Verification Boundary

This revision distinguishes **implemented behavior** from **standards conformance**. The RTL was statically reviewed against the programmer-visible requirements described in this datasheet and against the relevant RISC-V architectural concepts. Static source review can establish what the RTL decodes and exposes, but it cannot prove complete ISA, privileged-architecture, debug, AXI, or cJTAG compliance.

The following terminology is used throughout this document:

- **Implemented** — directly supported by the supplied RTL.
- **RISC-V architectural behavior** — behavior intended to follow the cited RISC-V ISA or privileged architecture.
- **Subset / implementation-specific** — only the documented functions are provided; software

must not assume unimplemented optional features.

- **Compliance not established** — requires directed testbenches, architectural compliance suites, protocol assertions, or external debugger interoperability tests.

Important implementation boundaries found during review:

- `axi_clic` is a JV32 local interrupt controller with CLINT-style timer/software-interrupt registers and a compact external-interrupt array. It is **not documented as a complete standard CLIC implementation**.
- The debug block implements a useful subset of RISC-V Debug Specification 1.0 behavior, but this datasheet does **not** claim formal Debug Spec compliance.
- `AMO_EN=0` makes all AMO-opcode instructions illegal, including LR.W and SC.W. Verified by `sw/amo_config` in both atomic configurations and by the disabled-decoder sweep; the RTL parameter comment matches this behavior.
- AXI4-Lite behavior is implementation-specific to the documented channels and ordering restrictions. Protocol compliance should be verified with AXI assertions/testbench coverage.

1.5. Default Configuration

The table below summarizes the principal RTL defaults. Parameters remain the authoritative source when an integration overrides these values.

Table 1. Default RTL Configuration

Item	Default	Notes
XLEN	32	32-bit RISC-V implementation.
Privilege mode	M-mode	The core implements M-mode operation.
Base ISA	RV32I	RV32E can be selected when enabled by the corresponding RTL parameter.
ISA extensions enabled by default	M, A, C, Zicsr, Zifencei, Zba, Zbb, Zbs, Zcmp	<code>RV32M_EN</code> , <code>AMO_EN</code> , <code>RV32B_EN</code> , and <code>ZCMP_EN</code> control parameterized extension groups. C, Zicsr, and Zifencei are implemented by the supplied core configuration.
System clock parameter	100 MHz	Functional default only; not a defined implementation Fmax.
External interrupts	16	Handled by the local interrupt-controller path.

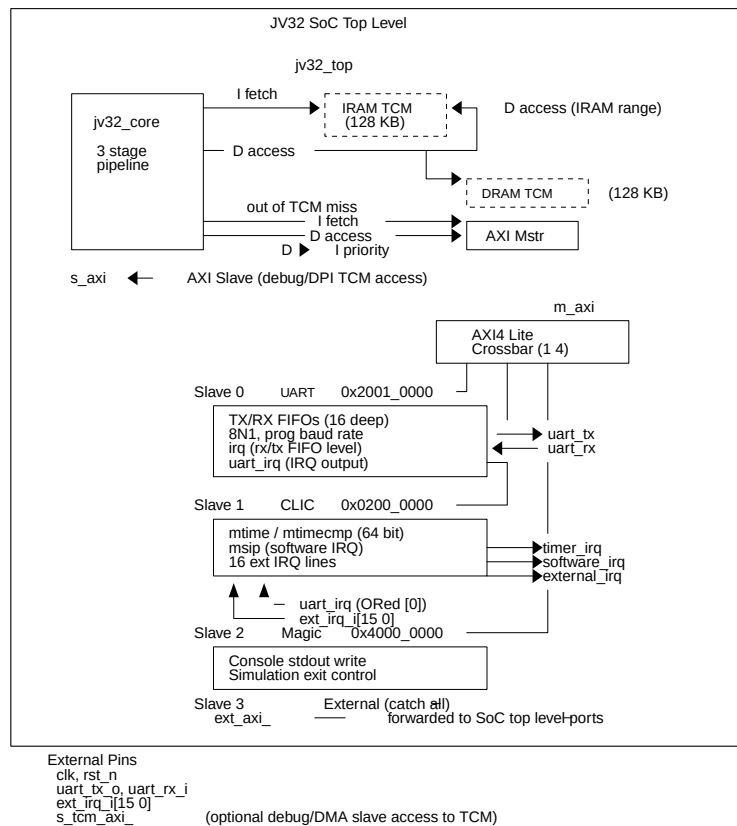
Item	Default	Notes
UART	115200 baud	Reset/default baud rate; software can change the divisor at runtime.

1.6. Key Features

Feature	Specification
ISA	Default configuration: RV32IMAC with Zicsr, Zifencei, Zba, Zbb, Zbs, and Zcmp enabled. <code>misa = 0x4000_1105</code> reports the standard MISA-visible I/M/A/C capabilities; Zba/Zbb/Zbs and Zcmp are controlled separately by RTL parameters.
Pipeline	Single-issue 3-stage in-order (Fetch, Execute, Writeback); WB → EX same-cycle forwarding; 1-cycle load-use stall; optional single-slot non-blocking MUL scoreboard (<code>SCOREBOARD_EN</code>); BTFNT static branch predictor (<code>BP_EN=1</code>); no EX-stage correction flush on correct prediction, one EX-stage correction/flush cycle on misprediction
Instruction Memory	128 KB IRAM (TCM, 1-cycle access); out-of-TCM I-fetch via shared AXI4-Lite master
Data Memory	128 KB DRAM (TCM, 1-cycle access); IRAM is also D-accessible (IRAM address range); out-of-TCM via shared AXI4-Lite master
Privilege Modes	Machine (M-mode) only
Bus Protocol	AXI4-Lite (32-bit address, 32-bit data)
Clock Frequency	100 MHz default; parameterized
Boot Address	<code>0x8000_0000</code> (IRAM base, parameterized)
TCM Sizes	128 KB IRAM, 128 KB DRAM (each independently parameterized, power-of-2)
Peripherals	UART (8N1, 16-deep FIFOs), local interrupt controller (CLINT-style timer/software interrupt plus 16 external IRQs), Magic (simulation I/O)
Interrupts	Machine timer, machine software, and 16 local-controller external interrupt lines (<code>ext_irq_i[15:0]</code>)
Multiply Latency	1 cycle (<code>FAST_MUL=1</code>), 1–32 cycles (<code>FAST_MUL=0</code> , CLZ-reduced on multiplier)
Divide Latency	1 cycle (<code>FAST_DIV=1</code>), 1–32 cycles (<code>FAST_DIV=0</code> , CLZ-reduced on dividend)
Implementation	RTL simulation (Verilator), FPGA synthesis supported

Chapter 2. System Architecture

2.1. Block Diagram



2.2. System Components

2.2.1. Processor and Memory Integration

The processor and integrated memory-mapped interfaces use little-endian byte ordering. Bit numbering follows the RISC-V convention: bit 0 is the least-significant bit. Addresses are byte addresses.

The **juv32_core** processor implements a single-issue 3-stage in-order pipeline:

- **Instruction Fetch (IF):** PC generation, RVC expansion, instruction fetch request
- **Execute (EX):** Decode, register read, ALU, address calculation, CSR operations, hazard detection
- **Writeback (WB):** Memory result, register writeback, branch redirect, exception handling

The core supports:

- Single-issue in-order execution; Slot-A all instructions; no dual-issue
- WB → EX same-cycle forwarding for RAW hazard resolution
- 1-cycle load-use stall (EX → WB stall + IF hold)
- 1-cycle branch/jump flush (EX squashes IF on misprediction or unconditional redirect)
- Multi-cycle stall for slow divide/shift and for multiply when scoreboard dispatch is not active
- Optional single-slot non-blocking scoreboard (**SCOREBOARD_EN**) for multi-cycle **MUL**: independent younger instructions may issue while preserving in-order retirement
- AMO (LR/SC and AMO operations) via a 3-state machine (IDLE → LOAD_WAIT → STORE_WAIT)
- Configurable branch predictor (**BP_EN**, static not-taken fallback when disabled)
- CLIC-aware CSR unit with CLIC interrupt-level threshold (**mintthresh**), current level (**mintstatus**), vectored-trap base (**mtvt**), next-interrupt fast-claim (**mnxti**), and hardware tail-chaining on **mret**

2.2.2. TCM: IRAM and DRAM (**jv32_top**)

Both TCM regions are instantiated inside **jv32_top** as a single 32-bit wide **sram_1rw** primitive per region, with byte-enable write support:

- **IRAM**: instance **u_iram**, base **IRAM_BASE** (default **0x8000_0000**), size **IRAM_SIZE** (default 128 KB); 32-bit data word, 4-bit write-byte-enable
- **DRAM**: instance **u_dram**, base **DRAM_BASE** (default **0x9000_0000**), size **DRAM_SIZE** (default 128 KB); same arrangement
- **1-cycle read latency**: Address presented at cycle N returns data at cycle N+1
- **Contention resolution**: When I-fetch and D-access both target IRAM, D-access wins; I-fetch stalls 1 cycle
- **SRAM arbitration priority**: Core-D > Core-I > AXI Slave

2.2.3. Merged AXI4-Lite Master

Out-of-TCM accesses from I-fetch and D-bus share one AXI4-Lite master (**m_axi_***):

- D-bus has priority over I-bus when both miss TCM simultaneously
- Both are served sequentially (not simultaneously)
- I-fetch AXI transaction is cancelled/replaced if a branch redirect arrives during the outstanding fetch

2.2.4. AXI4-Lite Slave

External agents (debug master, pre-simulation ELF loader via DPI) access TCM through the slave port (`s_axi_*`):

- Core has absolute priority; slave stalls until the target SRAM is idle for the current cycle
- The slave read-data register latches SRAM output on the first granted cycle and holds it until the bus handshake completes
- For pre-simulation ELF loading, the recommended method is DPI `mem_write_byte` (direct SRAM array access), which is faster than AXI slave transactions

2.2.5. Peripheral AXI Crossbar

A 1-to-4 AXI4-Lite crossbar routes out-of-TCM transactions from `jv32_top's merged master to three named peripheral slaves (UART, CLIC, Magic) plus a fourth external catch-all slave. Address decode uses base/mask comparison (see [Memory Map](#)).

2.3. Register Table Conventions

Register tables use byte offsets relative to the peripheral base address unless an absolute address is explicitly shown. Reset values describe the RTL reset state before software modifies the register. For reserved fields, software should write zero and ignore the read value unless a section explicitly defines different behavior.

Chapter 3. Memory Map

3.1. Address Space Overview

The JV32 SoC implements a 32-bit address space (4 GB). The table lists the default address map. TCM region sizes follow the `IRAM_SIZE` and `DRAM_SIZE` parameters. The `0x6000_0000` and `0x7000_0000` regions are aliases of the canonical IRAM and DRAM regions; they do not provide additional storage. Software should normally use the canonical `0x8000_0000` and `0x9000_0000` addresses unless an alias is specifically required by the integration or debug flow.

Table 2. Default Memory Map

Address Range	Device	Size	Description
<code>0x0200_0000</code> – <code>0x021F_FFFF</code>	CLIC	2 MB	Project-specific local interrupt controller with CLINT-style timer/software-interrupt offsets; only the implemented offsets in the first 64 KB are meaningful
<code>0x2001_0000</code> – <code>0x2001_00FF</code>	UART	256 B	Serial communication peripheral (8N1, 16-deep FIFOs); 7 registers used (0x00–0x18); 0x1C–0xFF return <code>SLVERR</code>
<code>0x4000_0000</code> – <code>0x4FFF_FFFF</code>	Magic Device	256 MB (aperture)	Simulation console and exit control; only two 32-bit registers are implemented at offsets <code>0x0000</code> and <code>0x0004</code>
<code>0x6000_0000</code> – <code>0x6001_FFFF</code>	IRAM alias	128 KB	Alias of the default IRAM TCM region; internally mapped to <code>0x8000_0000</code> – <code>0x8001_FFFF</code>
<code>0x7000_0000</code> – <code>0x7001_FFFF</code>	DRAM alias	128 KB	Alias of the default DRAM TCM region; internally mapped to <code>0x9000_0000</code> – <code>0x9001_FFFF</code>
<code>0x8000_0000</code> – <code>0x8001_FFFF</code>	IRAM (TCM)	128 KB	Tightly-coupled instruction + data RAM (inside <code>fv32_top</code>)
<code>0x9000_0000</code> – <code>0x9001_FFFF</code>	DRAM (TCM)	128 KB	Tightly-coupled data RAM (inside <code>fv32_top</code>)



The canonical TCM regions and their aliases are decoded before the peripheral crossbar. They are decoded inside `fv32_top` before the AXI master port. Only accesses that miss TCM are forwarded to the crossbar.

3.2. Address Decode Rules

3.2.1. TCM Decode (inside `jav32_top`)

TCM is decoded combinatorially using power-of-2 masking:

Table 3. TCM Address Decode Functions

Region	Condition	Default
IRAM	$(\text{addr} \ \& \ \sim(\text{IRAM_SIZE}-1)) == \text{IRAM_BASE}$	<code>IRAM_BASE=0x8000_0000, IRAM_SIZE=131072</code>
DRAM	$(\text{addr} \ \& \ \sim(\text{DRAM_SIZE}-1)) == \text{DRAM_BASE}$	<code>DRAM_BASE=0x9000_0000, DRAM_SIZE=131072</code>

The SoC also accepts the default alias windows at `0x6000_0000` (IRAM) and `0x7000_0000` (DRAM). Alias accesses are converted to the corresponding canonical TCM address. Accesses that do not match a canonical or alias TCM region are forwarded to the merged AXI master and peripheral crossbar.

3.2.2. Peripheral Crossbar Decode

The AXI crossbar (`axi_xbar`) decodes using base/mask:

Table 4. Peripheral Crossbar Address Decode

Slave	Device	Base	Mask	Notes
0	UART	<code>0x2001_00_00</code>	<code>0xFFFF_FF_00</code>	256-byte aperture; 7 registers (DATA, STATUS, IE, IS, LEVEL, CTRL, CAPABILITY at 0x00–0x18)
1	Local interrupt controller	<code>0x0200_00_00</code>	<code>0xFFE0_00_00</code>	2 MB aperture; CLINT-style timer/software registers plus implementation-specific external-interrupt array
2	Magic	<code>0x4000_00_00</code>	<code>0xF000_00_00</code>	256 MB aperture; only two 32-bit registers are implemented (<code>0x0000</code> and <code>0x0004</code>)
3	External catch-all	<code>0x0000_00_00</code>	<code>0x0000_00_00</code>	Matches all addresses not decoded by UART, CLIC, or Magic and forwards them to the <code>ext_axi_*</code> SoC ports. The final response is provided by the connected external target.

In the default SoC map, addresses outside the internal TCM and named peripheral regions are

forwarded to the external AXI target. They do not automatically produce **DECERR**; the external target determines the response.

Chapter 4. Processor Core Specification

4.1. Data Representation

The core uses 32-bit little-endian data representation. Bit 0 is the least-significant bit, and all addresses are byte addresses. Instruction alignment follows the enabled RISC-V compressed-instruction configuration.

4.2. ISA Support

4.3. RV32E Configuration

When `RV32E_EN=1`, the architectural integer register set is reduced to x0–x15. The decoder checks register fields according to the instruction format rather than blindly checking every encoded five-bit field:

- `rs1` is checked only for formats that actually use `rs1`.
- `rs2` is checked for OP, STORE, BRANCH, and AMO formats.
- `rd` is checked only when the decoded instruction writes a GPR.

An instruction that actually references x16–x31 is illegal. Immediate fields that happen to contain bit 4 in the same bit positions are not incorrectly rejected.

The core also forces Zba/Zbb/Zbs inactive and disables the RAS and instruction prefetch buffer when `RV32E_EN=1`. Compressed register-prime encodings use x8–x15 and remain within the RV32E register set.

Table 5. Supported ISA Extensions

Extension	Description
RV32I	Base integer instruction set (32-bit)
M	Integer multiply and divide, including MULH, MULHU, MULHSU, DIV, DIVU, REM, REMU
A	Atomic instructions: LR.W/SC.W, AMO{SWAP,ADD,XOR,AND,OR,MIN,MAX,MINU,MAXU}.W
C (Zca)	16-bit compressed base instructions; expanded to 32-bit by <code>jv32_rvc</code> before the Execute stage

Extension	Description
Zcb-style compressed operations	jv32_rvc also expands the implemented compressed byte/halfword load/store and compact ALU encodings controlled by its internal RVM23_EN=1 configuration. These operations are part of the supplied RTL configuration; individual expanded instructions still depend on the corresponding base/M capability being enabled.
Zcmp (optional, ZCMP_EN=1)	Implements cm.push , cm.pop , cm.popret , cm.popretz , cm.mvsa01 , and cm.mva01s ; enabled in the default configuration
Zicsr	CSR access instructions: CSRRW , CSRRS , CSRRC , CSRRWI , CSRRSI , CSRRCI
Zifencei	Instruction-fetch fence (FENCE.I): flushes the IF stage and re-fetches from PC+4
Zba (optional, RV32B_EN=1)	Address generation: SH1ADD , SH2ADD , SH3ADD — shift-and-add for scaled pointer arithmetic
Zbb (optional, RV32B_EN=1)	Basic bit manipulation: CLZ , CTZ , CPOP , ANDN , ORN , XNOR , MIN/MAX/MINU/MAXU , SEXT.B , SEXT.H , ZEXT.H , REV8 , ORC.B , ROL , ROR , RORI
Zbs (optional, RV32B_EN=1)	Single-bit instructions: BCLR , BEXT , BINV , BSET and their immediate variants (BCLRI , BEXTI , BINVI , BSETI)



M-mode (M-mode) only. User mode and Supervisor mode are not implemented.

4.4. Pipeline Architecture

The core uses a single-issue 3-stage in-order pipeline:

Table 6. Pipeline Stages

Stage	Name	Function
IF	Instruction Fetch	Generate PC; present address to TCM or AXI master; pass fetched instruction through jv32_rvc RVC expander
EX	Execute	Decode opcode; read register file; execute ALU/branch/load-address/store-address/CSR; detect hazards; compute branch target; AMO state machine
WB	Writeback	Receive memory response; write result to register file; handle exceptions; redirect pipeline on branch/jump/exception/MRET/interrupt

4.5. Forwarding and Hazards

Table 7. Pipeline Hazard Handling

Hazard	Type	Description
RAW (register read-after-write)	Forwarding	WB result forwarded to EX operand inputs in the same cycle. No stall required for ALU → ALU dependency.
Load-use	Stall (1 cycle)	Load in EX presents address; data arrives in WB. The following EX-stage instruction must stall 1 cycle waiting for WB to complete. IF is also held.
Branch / Jump misprediction	Flush (1 cycle)	When EX detects a taken branch or unconditional jump, the IF instruction in flight is squashed (1 pipeline bubble).
Multi-cycle ALU (mul/div/shift)	Stall / partial overlap	Without scoreboard dispatch, multi-cycle operations stall EX/IF until completion. With scoreboard active (SCOREBOARD_EN=1 and multiply actually multi-cycle), one outstanding MUL can run in a dedicated slot while younger independent, non-commit-sensitive instructions continue. DIV/REM remain blocking.
Scoreboard issue gating (MUL non-blocking path)	Issue stall (selective)	Issue is blocked on RAW (rs1/rs2 busy), WAW (rd busy), structural (slot already occupied), or commit-sensitive younger instruction while slot is active. Retirement remains strictly in-order through WB arbitration.
AMO (LR/SC, AMO operations)	Stall	Two-phase read-modify-write sequence: EX issues AXI read, waits for response (LOAD_WAIT), then issues AXI write, waits for B-channel (STORE_WAIT). Pipeline stalls throughout.
FENCE / FENCE.I	No-op / Fetch synchronization	FENCE is decoded but treated as a no-op. FENCE.I redirects to the sequential PC, flushes the RVC/instruction-prefetch path, discards an in-flight stale AXI instruction response, and suppresses the first stale registered TCM response after the redirect.



FENCE.I is the instruction-fetch synchronization mechanism implemented by this core. **fv32_top** contains a FENCE.I-specific stale-response suppressor so that an IRAM instruction modified by a preceding TCM store is re-fetched instead of consuming the registered pre-write SRAM response. **FENCE.I** does not add general data-memory ordering semantics; **FENCE** remains a no-op. **sw/fencei_smc** verifies that every store before **FENCE.I** — including sub-word writes and writes issued while the TCM store queue is full — is visible to the

subsequent instruction fetch. `sw/mem_ordering` verifies that `FENCE` never needs to add ordering on the current configuration (program order is already preserved across TCM→TCM and TCM→MMIO).



`WFI` is also decoded but treated as a no-op in `jv32_core`. It does not stall the hart or enter a physical low-power state.

4.6. Atomic Operation Semantics

When `AMO_EN=1`, the decoder accepts the AMO opcode with `funct3=010` and the core executes LR.W, SC.W, and the implemented AMO.W operations through the atomic state machine.

4.6.1. LR/SC Reservation

The RTL uses a single reservation consisting of `lr_valid` and the exact 32-bit address captured when LR.W completes successfully.

- LR.W returns the loaded word and records the LR address.
- SC.W succeeds only when `lr_valid=1` and the SC address exactly matches the saved LR address.
- Successful SC.W returns `0`.
- SC.W without a matching reservation returns `1` and does not issue the store.
- Executing **any** SC.W invalidates the reservation, whether it succeeds or fails. A failing SC.W (address mismatch, or no live reservation) clears `lr_valid` in the same cycle; a matching SC.W clears it when the store transaction completes. This matches the RISC-V A-extension requirement that "regardless of success or failure, executing an SC.W instruction invalidates any reservation held by this hart".
- Ordinary loads, stores, traps, interrupts, and unrelated AMO operations by the hart itself do not clear `lr_valid`. For a single-hart system this is a legal implementation choice; a later SC.W is still bounded because the first SC.W in program order clears the reservation.
- A debug-module memory write — SBA or an abstract Access Memory command — to the reserved word invalidates the reservation on the next cycle, so an SC.W paired with an LR.W that the debugger overwrote fails. The comparison is word-granular and on canonical (de-aliased) addresses.

`aq / rl` bits on atomics are decoded but do not change core behaviour: the in-order single-issue pipeline and store queue already retire memory and I/O operations in an order at least as strong

as every acquire/release case on this single-hart configuration (verified by `sw/mem_ordering`).

4.6.2. AMO Encoding Scope

The supported AMO.W operations are `AMOSWAP`, `AMOADD`, `AMOXOR`, `AMOAND`, `AMOODR`, `AMOMIN`, `AMOMAX`, `AMOMINU`, and `AMOMAXU`, plus LR.W and SC.W.

The decoder validates `AMO_EN`, requires `funct3=010`, and WARL-checks `funct5`: only the eleven architecturally defined encodings above are accepted. Every other `funct5` value under the AMO opcode raises an illegal-instruction exception and issues no memory transaction.

4.6.3. Atomic Alignment

All implemented atomic operations are word operations and require a naturally aligned 32-bit address. A misaligned LR.W / SC.W / AMO.W address is detected in the EX stage and raises an address-misaligned exception before any memory transaction is issued: cause 4 (load address misaligned) for LR.W, and cause 6 (store/AMO address misaligned) for SC.W and every AMO.W. `mtval` holds the faulting address.

4.7. Control-Flow Prediction

When `BP_EN=1` (default), the front end combines static BTFNT conditional-branch prediction, one-entry last-branch reuse, JAL predecode, and the optional Return Address Stack (RAS). `BP_EN=0` disables these speculative front-end redirects.

4.7.1. Conditional Branch Prediction

For a conditional branch, the front end applies the following prediction rules:

1. A valid one-entry last-branch (`L0`) hit for the same branch PC reuses the most recently resolved taken target.
2. Otherwise, a backward branch (negative B-immediate) is predicted taken.
3. A forward branch is predicted not taken.
4. EX resolves the actual branch condition. A mismatch between the prediction and the resolved result redirects the fetch PC and flushes the speculative path.

The L0 entry is updated whenever a branch resolves and records that branch PC, resolved target, and whether the branch was taken. This allows a recently observed forward-taken branch to be predicted taken even though the normal BTFNT rule predicts forward branches as not taken.

When **BP_EN=0**, conditional branches are predicted not taken and a taken branch redirects from EX.

4.7.2. JAL

The front end recognizes JAL after RVC expansion and computes its PC-relative target before the instruction reaches EX. When front-end prediction is active, the fetch path redirects directly to that target. EX still computes the architectural JAL target and link value.

The architectural link address is **PC + 2** for a compressed call expanded by the RVC front end and **PC + 4** for a 32-bit JAL/JALR instruction.

4.7.3. JALR

JALR computes the architectural target in EX as:

```
target = (rs1 + immediate) & ~1
```

Bit 0 of the target is therefore always cleared.

A general JALR is not target-predicted by BTFNT. Eligible return patterns can instead be predicted by the RAS. If a RAS prediction is absent or does not match the EX-computed target, EX redirects to the computed JALR target.

4.7.4. Return Address Stack

The RAS is active only when **BP_EN=1**, **RAS_EN=1**, and **RV32E_EN=0**. It is a two-entry circular structure.

- **Push:** JAL or JALR with **rd** equal to **x1 (ra)** or **x5 (t0)** pushes the sequential return address.
- **Pop:** JALR with **rs1** equal to **x1** or **x5** and **rd** not equal to either link register uses and removes the current RAS top when the stack is non-empty.
- **Push + pop:** a JALR pattern that both reads and writes a link register is treated as a coroutine-style operation; the current top entry is replaced while the RAS pointer is unchanged.
- **Empty RAS:** no speculative return redirect is made.
- **Non-speculative redirect:** exception, MRET, or interrupt redirect clears the RAS state.
- **Branch correction and FENCE.I:** these redirects do not clear the RAS.

A RAS prediction is only speculative. EX always computes the JALR target and corrects the fetch path if the predicted return address is wrong.

4.8. RTL Operation Timing

The following timing statements are derived from the current RTL. They are not benchmark results and do not define a system-level performance guarantee.

Table 8. RTL Operation Timing

Operation	RTL Behavior
TCM read	The SRAM uses a registered read path. A granted read returns data on the following clock cycle.
Load-use dependency	The dependent instruction incurs one pipeline stall cycle.
Conditional branch	A correct BTFNT prediction requires no EX-stage correction flush. A misprediction causes one EX-stage correction/flush cycle.
BP_EN=0 taken branch	A taken branch requires an EX-stage redirect because no taken prediction is made.
Multiply	FAST_MUL=1 selects the fast multiply datapath. FAST_MUL=0 selects the iterative implementation. MUL_MC further controls the fast multiply implementation.
Divide	FAST_DIV=1 selects the fast divide datapath. FAST_DIV=0 selects the iterative implementation.
Shift	FAST_SHIFT=1 selects the barrel shifter. FAST_SHIFT=0 selects the iterative shifter.
IRAM contention	A D-side IRAM access has priority over instruction fetch when both require the shared IRAM port.
External AXI transaction	No fixed latency is specified. Completion depends on the connected interconnect and target.

This datasheet does not define typical CPI, benchmark performance, interrupt-entry latency, or maximum clock frequency. Those values depend on the selected RTL parameters, software workload, memory placement, external-system latency, synthesis target, and physical implementation.

4.9. CSR (Control and Status Registers)

CSR instructions are checked by the decoder before execution. Access to an unknown CSR is illegal. A write attempt to a read-only CSR is illegal; CSRRS/CSRRC and their immediate forms remain read-only accesses when the source value is zero. The core implements only the CSR addresses listed in

this datasheet.

The core implements M-mode CSRs and CLIC-related CSRs. `misa` reports the MISA-visible base/standard extension bits implemented by `jv32_csr`; separately parameterized Zba/Zbb/Zbs and Zcmp support is not encoded in `misa`.

Table 9. Implemented CSRs

Address	Name	Access	Description
0x300	mstatus	R/W	Machine status: MIE[3], MPIE[7], MPP[12:11]=11 (M-mode always)
0x301	misa	R/O	ISA and extensions. Default value: <code>0x4000_1105</code> (RV32IMAC). The value changes with the configured ISA parameters.
0x304	mie	R/W	Machine interrupt-enable. Only MSIE[3], MTIE[7], and MEIE[11] are writable; other bits read as zero.
0x305	mtvec	R/W	Machine trap-handler base. The RTL preserves only mode bit 0: <code>0</code> = Direct and <code>1</code> = Vectored. Bit 1 is forced to zero. CLIC vectoring uses <code>mtvt</code> independently.
0x307	mtvt	R/W	CLIC vector table base address. The RTL forces bits [5:0] to zero, giving 64-byte alignment.
0x310	mstatush	R/O	Upper machine-status view; returns zero in this RV32 implementation
0x320	mcountinhibit	R/W	Counter control: CY[0] inhibits <code>mcycle</code> ; IR[2] inhibits <code>minstret</code>
0x340	mscratch	R/W	Machine scratch register (software use)
0x341	mepc	R/W	Machine exception program counter. Bit 0 is forced to zero on CSR writes.
0x342	mcause	R/W	Machine trap cause: bit[31]=interrupt flag, bits[30:0]=cause code
0x343	mtval	R/W	Machine trap value (faulting address for memory exceptions, 0 otherwise)

Address	Name	Access	Description
0x344	mip	R/O	Machine interrupt-pending: MSIP[3], MTIP[7], MEIP[11]
0x345	mnxti	R/W	CLIC next-interrupt: reads the vector address ($mtvt + 4 \cdot id$) of the highest-priority pending CLIC interrupt with level > mintthresh ; returns 0 if no such interrupt is pending. A write (e.g. csrrsi mnxti, 0) also atomically claims the interrupt: updates mcause , sets mintstatus.mil , and re-enables MIE , allowing the handler to branch directly to the returned address (software-assisted tail-chain).
0x347	mintthresh	R/W	CLIC interrupt threshold (8-bit): only CLIC interrupts with level > mintthresh are taken
0xB00	mcycle	R/W	Machine cycle counter (lower 32 bits)
0xB02	minstret	R/W	Machine instructions-retired counter (lower 32 bits)
0xB80	mcycleh	R/W	Machine cycle counter (upper 32 bits)
0xB82	minstreth	R/W	Machine instructions-retired counter (upper 32 bits)
0xC00	cycle	R/O	Unprivileged alias of mcycle (lower 32 bits)
0xC01	time	R/O	Unprivileged alias (lower 32 bits; maps to CLIC mtime)
0xC02	instret	R/O	Unprivileged alias of minstret (lower 32 bits)
0xC80	cycleh	R/O	Unprivileged alias of mcycleh
0xC81	timeh	R/O	Upper 32 bits of CLIC mtime
0xC82	instreth	R/O	Unprivileged alias of minstreth
0xF11	mvendorid	R/O	Vendor ID (returns 0x0000_0000)
0xF12	marchid	R/O	Architecture ID (returns 0x0000_0000)
0xF13	mimpid	R/O	Implementation ID (returns 0x0000_0001)

Address	Name	Access	Description
0xF14	mhartid	R/O	Hardware thread ID (returns 0x0000_0000)
0xFB1	mintstatus	R/O	CLIC interrupt status: bits[31:24] = mil (machine interrupt level currently servicing)

4.10. Exception and Interrupt Handling

4.10.1. Exception Types

The table lists exceptions that are actively generated by the current RTL. The core uses 16-bit instruction alignment when compressed instructions are enabled.

Table 10. Exception Causes (*mcause*[31]=0)

Code	Name	Description
1	Instruction access fault	I-fetch AXI error response (SLVERR or DECERR from the AXI master)
2	Illegal instruction	Undefined, unsupported, disabled-by-parameter, or otherwise rejected instruction encoding. mtval receives orig_instr ; for compressed instructions this preserves the original compressed encoding supplied by the RVC front end.
3	Breakpoint	EBREAK instruction executed
4	Load address misaligned	Load address not naturally aligned to the access size, including a misaligned LR.W address
5	Load access fault	Load AXI error response
6	Store address misaligned	Store address not naturally aligned to the access size, including a misaligned SC.W or AMO.W address
7	Store/AMO access fault	Store, SC, or AMO write-side AXI error response. Atomic read-side failures are reported through the memory-fault path based on the active AMO transaction phase.
11	Environment call from M-mode	ECALL executed in M-mode

Instruction-address-misaligned exceptions cannot arise from ordinary control transfers in the supported configurations: compressed instructions are always present (**IALIGN=16**), JAL/branches

encode halfword offsets, and JALR clears bit 0. Integrators must provide a halfword-aligned reset vector. The CSR test separately verifies cause-0 trap-state capture.



The EX-stage misalignment check covers normal loads and stores and, in this RTL, also LR.W / SC.W / AMO.W. A misaligned atomic raises cause 4 (LR.W) or cause 6 (SC.W, AMO.W) with `mtval` = faulting address, and no memory transaction is issued.

4.10.2. Exception Detection Priority

For an instruction reaching EX, the core checks synchronous conditions in this order:

1. instruction-fetch access fault;
2. illegal instruction (includes reserved AMO `funct5` encodings);
3. EBREAK that is not redirected into Debug Mode;
4. ECALL;
5. load/store/AMO address misalignment (normal loads and stores, and LR.W / SC.W / AMO.W).

The first matching condition is recorded in the EX/WB pipeline state. Data-memory AXI faults are detected later in WB and become load/store access-fault exceptions. Because illegal-instruction ranks above the misalignment check, a reserved AMO encoding with a misaligned address reports illegal instruction.

4.10.3. Redirect and Trap Priority

At the WB redirect point, the current RTL applies the following priority:

1. synchronous exception or data-memory access fault;
2. MRET;
3. pending interrupt;
4. EX-stage control-flow redirect such as a branch correction, JALR correction, or FENCE.I.

A pending interrupt therefore does not override an older synchronous exception or an MRET already in WB. When an interrupt is accepted, the WB instruction is canceled rather than retired; its PC is saved as the interrupt `mepc` so that it is re-executed after MRET.

During single-step execution, normal interrupt acceptance is suppressed while the step is pending. CLIC MRET tail chaining is handled separately: if MRET executes while a CLIC interrupt above

`mintthresh` is pending, the core redirects directly to the next CLIC vector instead of first returning to `mepc`.

4.10.4. Interrupt Types

Table 11. Interrupt Causes (`mcause[31]=1`)

Code	Name	Source
3	Machine software interrupt	CLIC MSIP register (<code>msip[0]</code>)
7	Machine timer interrupt	CLIC: <code>mtime >= mtimecmp</code>
11	Machine external interrupt	CLIC: any enabled pending external IRQ (<code>ext_irq_i[15:0]</code>) with level > <code>mintthresh</code>

4.10.5. Trap Handling Sequence

On exception or interrupt:

1. `mstatus.MPIE` $\rightarrow$ `mstatus.MIE`; `mstatus.MIE` $\rightarrow$ 0 (disable interrupts)
2. `mstatus.MPP` $\rightarrow$ 2'b11 (M-mode)
3. `mepc` $\rightarrow$ PC of interrupted/excepted instruction
4. `mcause` $\rightarrow$ {1'b1/1'b0, `cause_code`}
5. `mtval` $\rightarrow$ `faulting_address` (or 0)
6. PC redirected to `mtvec` (direct mode) or `mtvec + 4*cause` (vectored mode)

CLIC vectored interrupts redirect to `mtvt + 4*cllic_id` (vector table indexed by the winning interrupt's ID) and set `mintstatus.mil` to the claimed interrupt level.

Normal MRET restores: `mstatus.MIE` $\rightarrow$ `mstatus.MPIE`; `mstatus.MPIE` $\rightarrow$ 1; PC $\leftarrow$ `mepc`.

4.10.6. Tail-Chaining (Hardware)

If, at the moment `mret` retires, a CLIC external interrupt is pending with level > `mintthresh`, the core performs a **tail-chain** rather than a full context restore:

- Fetch is redirected to `mtvt + 4*cllic_id` (the next handler) — zero overhead cycles.
- `mstatus.MIE` remains 0 (already in a new handler).
- `mstatus.MPIE` is set to 1 (so the eventual chain-terminating `mret` re-enables MIE).

- `mepc` is unchanged (still holds the preempted task's return address).
- `mcause` is updated to `0x8000_000B` (machine external interrupt).
- `mintstatus.mil` is updated to the new interrupt's level.

Tail-chaining eliminates the push/pop overhead between consecutive ISRs of equal or lower priority and is fully transparent to software: the same trap handler is re-entered for each chained interrupt.

4.10.7. Software-Assisted Tail-Chain (`mnxti`)

Within a CLIC handler, software can voluntarily tail-chain to the next pending interrupt by reading `mnxti`:

```
uintptr_t next_pc = csr_swap(mnxti, 0);
if (next_pc) goto (void *)next_pc; // chain to next handler
```

When `mnxti` is written (any write, e.g. `csrrsi mnxti, 0`) and a qualifying interrupt is pending:

- `mcause` and `mintstatus.mil` are atomically updated to the new interrupt.
- `mstatus.MIE` is re-enabled (nested interrupt within the handler).
- The read value is `mtvt + 4*clic_id` of the newly claimed interrupt (or 0 if none).

Software branches to the returned address to begin serving the next interrupt without an `mret`/re-entry cycle.

Chapter 5. TCM Specification

5.1. Overview

Both IRAM and DRAM are physically located inside `jv32_top` as a single 32-bit wide `sram_1rw` instance per region, with 4-bit byte-enable write support:

Table 12. TCM Configuration

Parameter	Default Value
IRAM base address	<code>IRAM_BASE = 32'h8000_0000</code>
IRAM size	<code>IRAM_SIZE = 131072</code> bytes (128 KB); power-of-2
DRAM base address	<code>DRAM_BASE = 32'h9000_0000</code>
DRAM size	<code>DRAM_SIZE = 131072</code> bytes (128 KB); power-of-2
Read latency	1 cycle (registered output: addr at cycle N → data at cycle N+1)
Write behavior	Selected byte lanes are written on the active clock edge. <code>sram_1rw</code> uses NO_CHANGE read-during-write behavior: the registered read output is not updated on a write cycle.
Data width	32 bits; 4-bit byte-enable write (<code>we[3:0]</code>); each byte lane independently writable
DPI hierarchy (IRAM)	<code>u_soc.u_jv32.u_iram.mem[word_index]</code> (byte access: <code>[byte_lane*8+:8]</code> , lane = 0..3)
DPI hierarchy (DRAM)	<code>u_soc.u_jv32.u_dram.mem[word_index]</code> (byte access: <code>[byte_lane*8+:8]</code> , lane = 0..3)

5.2. Access Priorities

When multiple requestors target the same SRAM in the same cycle:

Table 13. SRAM Arbitration Priority

Priority	Requestor	Notes
1 (highest)	Core D-path	D-access (load, store, AMO) always wins
2	Core I-path (I-fetch)	If D-path does not need the SRAM this cycle

Priority	Requestor	Notes
3 (lowest)	AXI Slave	Only granted when neither I-path nor D-path accesses the target SRAM

5.3. TCM Timing

Table 14. Timing Summary

Transaction	Behaviour
I-fetch (IRAM hit)	Addr presented at cycle N; instruction available at cycle N+1 (1-cycle latency)
D-read (IRAM or DRAM hit)	Addr presented at cycle N; data available at cycle N+1; pipeline inserts 1-cycle load stall
Core D-write (IRAM or DRAM hit)	Selected byte lanes are written on the active clock edge. This is a core-local TCM access, not an AXI transaction.
IRAM contention (D + I both need IRAM)	D-path wins; I-fetch stalls 1 extra cycle
Stale I-fetch after redirect	<code>imem_flush</code> signal (= <code>rvc_flush</code>) gates the TCM response; cycle N+1 response is suppressed when a branch/exception redirect occurred at cycle N

5.4. TCM AXI Slave Semantics

The TCM slave path uses `axi_ram_ctrl` and the shared single-port SRAM.

- AW and W may arrive independently; each channel is held until both parts of the write transaction are available.
- `WSTRB[3:0]` directly controls the four SRAM byte write enables.
- The address presented to SRAM is `addr >> 2`; low address bits `[1:0]` are not checked. Software and external masters must therefore issue naturally aligned accesses.
- A read returns AXI `OKAY`; no address-alignment error is generated by `axi_ram_ctrl`.
- If a RAM controller instance is configured with `WR_EN=0`, writes return `SLVERR`; writable TCM instances return `OKAY`.
- SRAM contents are not initialized by `sram_1rw` reset. Program/data initialization is supplied by the simulation/FPGA integration flow.

5.5. ELF Loading (Pre-Simulation)

For Verilator simulation, program images are loaded directly into TCM SRAM arrays via DPI-C before simulation starts. The testbench `tb_jv32_soc.cpp` calls `load_program()` from `elfloader.cpp`:

```
// Load ELF into TCM via DPI mem_write_byte
void mem_write_byte(uint32_t addr, uint8_t data) {
    if (addr >= IRAM_BASE && addr < IRAM_BASE + IRAM_SIZE) {
        uint32_t offset = addr - IRAM_BASE;
        uint32_t word    = offset >> 2;
        uint32_t bank    = offset & 3;
        // Writes: u_soc.u_jv32.u_iram.mem[word][bank*8+:8]
    } else if (addr >= DRAM_BASE && addr < DRAM_BASE + DRAM_SIZE) {
        uint32_t offset = addr - DRAM_BASE;
        uint32_t word    = offset >> 2;
        uint32_t bank    = offset & 3;
        // Writes: u_soc.u_jv32.u_dram.mem[word][bank*8+:8]
    }
}
```

Chapter 6. Peripheral Specifications

6.1. UART (Universal Asynchronous Receiver/Transmitter)

6.1.1. UART Overview

The UART peripheral provides serial communication with configurable baud rate and 8N1 format.

Table 15. UART Features

Feature	Specification
Base Address	0x2001_0000
Aperture	256 B (0x2001_0000–0x2001_00FF); valid register offsets are 0x00–0x18. Accesses from 0x1C to 0xFF return SLVERR.
Data Format	8 bits, no parity, 1 stop bit (8N1)
Baud Rate	The reset/default baud rate is selected by CLK_FREQ and BAUD_RATE. Software can change the runtime divisor by writing offset 0x10.
TX FIFO Depth	16 entries (parameterized: FIFO_DEPTH, must be power-of-2)
RX FIFO Depth	16 entries (same as TX)
Interrupt	Level-triggered; asserted when (IE & IS) != 0

6.1.2. UART Register Map

Reset clears IE, CTRL, and FIFO counts. STATUS reads zero and IS reads 0x2 (TX empty). BAUDDIV resets to CLK_FREQ / BAUD_RATE - 1; CAPABILITY reflects the configured FIFO depth and version 1. Reading an empty DATA register does not provide a defined byte.

Table 16. UART Registers (base 0x2001_0000)

Offset	Name	Access	Description
0x00	DATA	R/W	Write: pushes <code>wdata[7:0]</code> to the TX FIFO only when <code>WSTRB[0]=1</code> and the FIFO is not full. A write while the FIFO is full is silently dropped and still receives <code>OKAY</code>. Read: returns the byte at the RX FIFO read pointer and pops it only when the RX FIFO is non-empty. Software must check <code>STATUS.rx_ready</code> before reading DATA; the value returned while the FIFO is empty is not a defined data value.
0x04	STATUS	R/O	[0] tx_busy — TX FIFO full / cannot accept data (same signal as <code>tx_full</code>) [1] tx_full — TX FIFO full flag [2] rx_ready — RX FIFO has data available (not empty) [3] rx_full — RX FIFO full. An incoming received byte is silently dropped while this condition is true. NOTE: bits[1:0] reflect the same underlying <code>txf_full</code> signal. Bit 0 is therefore not a transmitter-shift-register busy indication; it only indicates that the TX FIFO cannot accept another byte.
0x08	IE (Interrupt Enable)	R/W	[0] rx_ready_ie — enable RX-ready interrupt [1] tx_empty_ie — enable TX empty interrupt. Only <code>WSTRB[0]</code> enables an IE write; the implemented value is <code>wdata[1:0]</code>.
0x0C	IS (Interrupt Status)	R/O	[0] rx_ready — RX FIFO not empty (level-triggered) [1] tx_empty — TX FIFO empty (level-triggered)
0x10	LEVEL / BAUDDIV	R/W	Read: [15:0] rx_count — number of bytes in RX FIFO; [31:16] tx_count — number of bytes in TX FIFO. Write: [15:0] runtime baud-rate divisor (<code>CLKS_PER_BIT - 1</code>); upper 16 bits are ignored. <code>WSTRB[0]</code> enables bits [7:0] and <code>WSTRB[1]</code> enables bits [15:8]; a zero-strobe write leaves the divisor unchanged. The divisor takes effect immediately, so changing it during an active TX or RX character can corrupt that character.

Offset	Name	Access	Description
0x14	CTRL	R/W	[0] <code>loopback_en</code> — internal TX → RX loopback. Only <code>WSTRB[0]</code> enables a CTRL write.
0x18	CAPABILITY	R/O	[7:0] TX FIFO depth (= <code>FIFO_DEPTH</code> parameter, default 16) [15:8] RX FIFO depth (= <code>FIFO_DEPTH</code> parameter, default 16) [31:16] UART version (<code>0x0001</code>); combined: <code>{16'h0001, 8'(FIFO_DEPTH), 8'(FIFO_DEPTH)}</code>



Interrupt asserts when `(IE & IS) != 0`. There is no explicit interrupt-clear register; the interrupt de-asserts automatically when the FIFO condition clears (e.g. RX FIFO drained, TX FIFO refilled).



The UART performs range checking using address bits `[7:2]`. Software should use the documented naturally aligned word offsets. Misaligned addresses within a valid word range are not part of the programming interface and may return `OKAY` without accessing the intended register.

6.1.3. Baud Rate Configuration

The reset value of the baud-rate divisor is derived at elaboration time:

```
CLKS_PER_BIT = CLK_FREQ / BAUD_RATE
baud_div_r   = CLKS_PER_BIT - 1
```

After reset, software may change `baud_div_r` by writing the lower 16 bits of UART offset `0x10`. The new bit period is `baud_div_r + 1` system-clock cycles.

RTL elaboration constraint: `CLKS_PER_BIT >= 4`. The non-synthesis assertion in `axi_uart.sv` rejects smaller values.

This datasheet does not define a production baud-rate margin beyond that RTL constraint. Board-level clock tolerance, UART peer tolerance, and implementation timing must be verified for the target system.

Table 17. Common Baud Rates at 100 MHz

Baud Rate	CLKS_PER_BIT	Notes
9600	10417	Very slow, large margin
115200	868	Standard; large margin
921600	109	Fast; good margin
1 Mbaud	100	Round number; good margin
12.5 Mbaud	8	Example configuration
25 Mbaud	4	Minimum divisor accepted by the RTL elaboration check



Configurations with **CLKS_PER_BIT** < 4 violate the RTL elaboration constraint.

6.1.4. Interrupt Behavior

- **RX interrupt** (**IS[0]**): Fires whenever RX FIFO is non-empty. Stays asserted until all bytes are consumed.
- **TX interrupt** (**IS[1]**): Fires whenever TX FIFO is empty. Stays asserted until software writes more data.

6.1.5. External Pins

Table 18. UART Pins

Signal	Direction	Description
uart_tx_o	Output	UART transmit data
uart_rx_i	Input	UART receive data (asynchronous to system clock)

6.2. Local Interrupt Controller (**axi_clic**)

6.2.1. Overview

The **axi_clic** block combines CLINT-style timer/software-interrupt registers with a 16-line implementation-specific external interrupt controller. It provides the internal sideband signals required by JV32 (**clic_irq**, **clic_id**, **clic_level**, and **clic_prio**), but it does not implement the complete standard RISC-V CLIC memory-mapped register model. Software should use the

register map defined in this datasheet, not a generic CLIC driver.

Table 19. CLIC Features

Feature	Specification
Base Address	0x0200_0000
Aperture	2 MB crossbar aperture. The RTL implements MSIP, MTIME, MTIMECMP, and NUM_IRQ CLICINT words; with the default NUM_IRQ=16 , only CLICINT[0]–CLICINT[15] are implemented.
Timer/software-interrupt layout	Implements the JV32 msip , mtime , and mtimecmp offsets documented below; treat these as the platform ABI for this RTL
Timer Counter	64-bit free-running mtime , increments every clock cycle
Timer Compare	64-bit mtimecmp ; timer IRQ fires when mtime $\geq$ mtimecmp
Software Interrupt	msip [0]: write 1 to assert, write 0 to clear
External Interrupt Lines	16 (NUM_IRQ=16), routed from ext_irq_i [15:0], each with individual enable/level/priority via CLICINT [n]
Interrupt Outputs	timer_irq_o , software_irq_o , cllic_irq_o (aggregated external), cllic_level_o [7:0], cllic_prio_o [7:0], cllic_id_o [4:0] (winning IRQ index; used by CPU to compute mtvt + 4*id)

6.2.2. Register Map

Table 20. Local Interrupt Controller Registers (base **0x0200_0000**)

Offset	Name	Access	Description
0x0000	MSIP	R/W	Machine Software Interrupt Pending. [0]= msip : write 1 to assert software interrupt; write 0 to clear. A write changes this bit only when WSTRB [0]=1; other byte lanes and zero-strobe writes leave it unchanged. Verified by tb_cllic .
0x4000	MTIME_LO	R/W	mtime [31:0] — lower 32 bits of 64-bit free-running timer
0x4004	MTIME_HI	R/W	mtime [63:32] — upper 32 bits
0x4008	MTIMECMP_LO	R/W	mtimecmp [31:0] — lower 32 bits of timer compare

Offset	Name	Access	Description
0x400C	MTIMECMP_HI	R/W	<code>mtimecmp[63:32]</code> — upper 32 bits. Timer IRQ fires when <code>mtime >= mtimecmp</code> .
0x1000 + n×4	CLICINT[n] (n=0..15)	R/W	Per-interrupt control word (32-bit): [0] <code>ip</code> (read-only) — interrupt pending; reflects <code>ext_irq_i[n]</code> (ORed with <code>uart_irq</code> for n=0) [1] <code>ie</code> — interrupt enable (1=enabled) [7:2] reserved — read as 0, writes ignored [15:8] reserved — read as 0, writes ignored (CLIC <code>attr</code> field not implemented in this RTL) [23:16] <code>ctl</code> — interrupt control value (8-bit; higher value wins arbitration and is also reported as the current level/priority) [31:24] reserved — read as 0. Writes to <code>ie</code> are applied only when <code>WSTRB[0]=1</code> ; writes to <code>ctl</code> are applied only when <code>WSTRB[2]=1</code> .



MSIP, MTIME, and MTIMECMP use the familiar single-hart CLINT-style offsets used by this design.



`axi_clic` always returns AXI `OKAY` for reads and writes. Unsupported offsets read as zero and writes have no effect. Software therefore must not use the AXI response code to discover whether a CLIC offset is implemented.

6.2.3. Timer Operation

The 64-bit `mtime` counter increments by 1 every system clock cycle. The timer interrupt fires when `mtime >= mtimecmp` (unsigned 64-bit comparison), **unless `mtimecmp` equals its reset sentinel value `0xFFFF_FFFF_FFFF_FFFF`** (startup guard — prevents a spurious interrupt before software programs a compare value). Software clears the timer interrupt by writing a new, larger value to `mtimecmp`. MTIME and MTIMECMP writes honor byte strobes. A zero-strobe MTIME write does not pause the counter. These behaviors are checked by `tb_clic`.

```
// Coherent 64-bit read on RV32; retry if the low word rolls over.
uint32_t hi, lo, hi_again;
do {
    hi = CLIC_MTIME_HI;
    lo = CLIC_MTIME_LO;
    hi_again = CLIC_MTIME_HI;
} while (hi != hi_again);
uint64_t now = ((uint64_t)hi << 32) | lo;
```

```
uint64_t cmp = now + 100000; // 1 ms at 100 MHz
// Comparator write order exercised by tb_clic:
CLIC_MTIMECMP_LO = 0xFFFFFFFFu;
CLIC_MTIMECMP_HI = (uint32_t)(cmp >> 32);
CLIC_MTIMECMP_LO = (uint32_t)cmp;
```

Use volatile MMIO accesses. The low-word sentinel, high-word update, and final low-word update avoid exposing a partially assembled compare value. Changing an already expired deadline does not erase an interrupt already accepted by the processor.

6.2.4. Software Interrupt Operation

Write `MSIP[0] = 1` to assert machine software interrupt immediately, `0` to clear.

6.2.5. External Interrupt Operation

Each `ext_irq_i[n]` input is gateable via `CLICINT[n].ie`. When any enabled and pending external interrupt has `ctl` (priority level) greater than the current `mintthresh` CSR value, `cllic_irq_o` is asserted to the core as an M-mode external interrupt.

When a CLIC interrupt is eligible (`cllic_irq` asserted above `mintthresh`) and global machine interrupts are enabled, the CSR interrupt selector gives the CLIC path precedence over the conventional MEI/MTI/MSI inputs. The CLIC asserts `cllic_id_o[4:0]` with the index of the winning pending enabled interrupt. Arbitration scans interrupt IDs in ascending order and replaces the current winner only when a strictly greater `cllicint_ctl` value is found. Therefore, when two pending interrupts have the same `ctl` value, the lower interrupt ID wins. Both `cllic_level_o[7:0]` and `cllic_prio_o[7:0]` carry the same `cllicint_ctl[winner]` value; this RTL does not implement separate level and priority fields. The CPU uses this to compute the handler address as `mtvt + 4*cllic_id` (where `mtvt` is programmed to a 4-byte-pointer table). This happens automatically on interrupt entry and on hardware tail-chain at `mret`; software can also perform the same lookup by reading `mnxti`.



UART IRQ routing: the UART `irq` output (asserted when `(IE & IS) != 0`) is OR'd into `ext_irq_i[0]` inside `lv32_soc` before being presented to the CLIC. The effective `CLICINT[0].ip` thus reflects `ext_irq_i[0] | uart_irq`. Software must enable `CLICINT[0].ie` and configure `CLICINT[0].ctl` to receive UART interrupts. The external `ext_irq_i[0]` SoC pin is still available and is ORed with the UART interrupt.

6.3. Magic Device (Simulation Only)

6.3.1. Magic Device Overview

The Magic Device provides simulation console I/O and simulation exit control.



The Magic Device is for simulation only. It must not be synthesized for FPGA/ASIC production.

Table 21. Magic Device Features

Feature	Specification
Base Address	0x4000_0000
Console Write	Offset 0x0000: write byte (bits[7:0]) to host <code>stdout</code>
Exit Control	Offset 0x0004: HTIF-encoded exit. Write <code>(exit_code << 1) 1</code> to exit with the given code; <code>1</code> = pass (exit code 0). The RTL calls <code>sim_request_exit(value >> 1)</code> . Writing <code>0</code> is ignored.

6.3.2. Magic Device Register Map

Table 22. Magic Device Registers (base 0x4000_0000)

Offset	Name	Access	Description
0x0000	CONSOLE	W/O	Write character (low byte) to simulation <code>stdout</code> . Reads return 0.
0x0004	EXIT	W/O	HTIF-encoded exit control: <code>sim_request_exit(value >> 1)</code> is called when any non-zero value is written. Write <code>1</code> = pass (exit code 0); write <code>(code << 1) 1</code> = fail with <code>code</code> . Writing <code>0</code> has no effect.

6.3.3. AXI Response Behavior

`axi_magic` compares addresses after clearing bits `[1:0]`; byte addresses within the 32-bit CONSOLE or EXIT word therefore select the same magic register. Recognized reads return zero with `OKAY`; unrecognized addresses return zero with `SLVERR`.

For writes, a one-hot `WSTRB` selects the corresponding byte from `WDATA`. Other strobe patterns use `WDATA[7:0]` for console-character extraction. EXIT uses the selected byte for one-hot byte writes and the full 32-bit `WDATA` for other write-strobe patterns.

6.3.4. Usage Example

```
#define MAGIC_CONSOLE (*(volatile uint32_t *)0x40000000)
#define MAGIC_EXIT    (*(volatile uint32_t *)0x40000004)

void print_str(const char *s) {
    while (*s) MAGIC_CONSOLE = *s++;
}

void sim_pass(void) { MAGIC_EXIT = 1; } // exit(0)
void sim_fail(int code) { MAGIC_EXIT = (code << 1) | 1; }
```

Chapter 7. AXI Interconnect and Bus Protocol

7.1. Protocol Scope

The integrated AXI interfaces implement the transaction behavior exposed by the supplied RTL. This datasheet documents the supported integration contract and does not imply support for AXI features that are absent from the module ports or control logic.

7.2. AXI4-Lite Overview

The JV32 SoC uses AXI4-Lite throughout (32-bit address, 32-bit data, no burst, no ID).

Table 23. AXI Protocol Summary

Feature	Specification
Protocol	AXI4-Lite (AMBA 4 subset: single-beat, no ARLEN/AWLEN/ARBURST, no IDs)
Data Width	32 bits
Address Width	32 bits
Write Strobes	4 bits (byte granularity)
Response Codes	OKAY (2'b00), SLVERR (2'b10), DECERR (2'b11)
Crossbar Topology	1 master and 4 decode targets in jv32_soc : UART, CLIC, Magic, and the external AXI catch-all target

7.3. **jv32_top** AXI Master Port (**m_axi_***)

The merged AXI master port carries all out-of-TCM traffic:

Table 24. AXI4-Lite Master Signals

Signal	Direction	Description
m_axi_araddr [31:0]	Output	Read address channel: address
m_axi_arvalid id	Output	Read address valid
m_axi_arready dy	Input	Read address ready (slave accepts)
m_axi_rdata [31:0]	Input	Read data

Signal	Direction	Description
m_axi_rresp[1:0]	Input	Read response (OKAY/SLVERR/DECERR)
m_axi_rvalid	Input	Read data valid
m_axi_rready	Output	Read data ready (master accepts)
m_axi_awaddr[31:0]	Output	Write address
m_axi_awvalid	Output	Write address valid
m_axi_awready	Input	Write address ready
m_axi_wdata[31:0]	Output	Write data
m_axi_wstrb[3:0]	Output	Write byte strobes
m_axi_wvalid	Output	Write data valid
m_axi_wready	Input	Write data ready
m_axi_bresp[1:0]	Input	Write response
m_axi_bvalid	Input	Write response valid
m_axi_bready	Output	Write response ready

7.4. jv32_top AXI Slave Port (s_axi_*)

The TCM slave port has the same signal set as the master port (with directions flipped). All s_axi_* signals are exposed as SoC-level ports (s_tcm_* prefix in jv32_soc).

7.5. Crossbar Architecture

The axi_xbar module implements a 1-to-N address-decoder crossbar:

- **Address decode:** $(addr \ \& \ SLAVE_MASK[i]) == (SLAVE_BASE[i] \ \& \ SLAVE_MASK[i])$ for

each slave in order; first match wins

- **Decode ordering:** slave indices are checked in ascending order and the first matching base/mask entry wins.
- **Crossbar-local decode error:** if no configured slave entry matches, the crossbar returns **DECERR** on both reads and writes.
- **SoC catch-all target:** the fourth `jv32_soc` decode entry uses base `0x0000_0000` and mask `0x0000_0000`, so every address not claimed by UART, CLIC, or Magic matches the external AXI target. In the default SoC integration, crossbar-local **DECERR** is therefore normally unreachable; the external target supplies the final response.
- **Protocol scope:** the exposed interconnect is AXI4-Lite and carries only single-beat transactions; there are no burst-length, burst-type, ID, or **RLAST** ports.

7.6. Bus State Machines (inside `jv32_top`)

The merged AXI master is implemented as a state machine with the following states:

Table 25. AXI Master State Machine

State	Description
BUS_IDLE	Both I-fetch and D-access in TCM or idle. In IDLE, D-bus out-of-TCM check has priority over I-bus.
DAR	D-bus AXI read address phase: <code>m_axi_araddr</code> set to D-bus address; wait for arready .
DR	D-bus AXI read data phase: wait for <code>m_axi_rvalid</code> ; return data to core D-path.
DAW	D-bus AXI write address+data phase: issue both AW and W channels (simultaneous when possible).
DB	D-bus AXI write response: wait for <code>m_axi_bvalid</code> .
IAR	I-bus AXI read address phase: <code>m_axi_araddr</code> set to I-fetch PC; wait for arready .
IR	I-bus AXI read data phase: wait for <code>m_axi_rvalid</code> ; return instruction to core I-path.

Chapter 8. Debug and JTAG

8.1. Debug Architecture

When `JTAG_EN=1`, the SoC instantiates the JTAG TAP, RISC-V Debug Transport Module (DTM), Debug Module control logic, abstract-command engine, System Bus Access (SBA), and hardware trigger support.

The Debug Module implements a **RISC-V Debug Specification 1.0-oriented subset**. Static RTL review confirms the implemented register encodings and major flows listed below, but full specification conformance requires dedicated debug-compliance testing. In the current RTL, `dmstatus.version` reads 3 and `dtmcs.version` reads 1 (the DTM encoding shared by spec 0.13 and 1.0). Optional 1.0 features that JV32 does not implement read back as "absent": `dmstatus.hasresethaltreq` = 0 (no halt-on-reset), `dmstatus.confstrptrvalid` = 0 (no configuration string), `dtmcs.errinfo` = 0, and the `dmcs2/hawindow/confstrptr0-3` registers return 0. `dmstatus.ndmresetpending` tracks `dmcontrol.ndmreset`; `dmstatus.stickyunavail` = 0 (the single hart is always present). `nextdm` (DMI 0x1d) reads 0 — this is the only Debug Module.

The implementation spans two clock domains:

- The TAP and DMI register bank operate in the TAP clock domain.
- Debug command execution, register access, memory access, and core control operate in the system-clock domain.
- Clock-domain crossings use synchronizers, toggle handshakes, or stable-payload transfer mechanisms implemented in `jtag_tap.sv` and `jv32_dtm.sv`.

When `JTAG_EN=0`, the debug logic is not instantiated and the associated debug outputs are tied to inactive values.

8.2. JTAG TAP

The TAP uses a 5-bit instruction register.

Table 26. Supported JTAG Instructions

Instruction	IR Value	Behavior
IDCODE	0x01	Shifts the 32-bit <code>IDCODE</code> value. The default RTL parameter is <code>0x1DEAD3FF</code> .

Instruction	IR Value	Behavior
DTMCS	0x10	Accesses the Debug Transport Module Control and Status register.
DMI	0x11	Accesses the 41-bit DMI scan register: 7-bit address, 32-bit data, and 2-bit operation.
BYPASS	0x1F	Selects the one-bit bypass register.
Other IR values	—	Return zero on TDO and do not dispatch DMI operations. Only IR 0x1F selects the one-bit BYPASS register.

After TAP reset, the instruction register selects IDCODE.

8.3. DTMCS

The implemented DTMCS value reports:

Table 27. DTMCS Fields

Field	Value	Description
version	1	DTM encoding used by RISC-V Debug Spec 0.13 and 1.0.
errinfo	0	Optional; extended DMI error detail is not implemented.
abits	7	DMI uses a 7-bit address field.
dmistat	0	No persistent DMI status error is reported through this field.
idle	1	The debugger should provide at least one Run-Test/Idle TCK cycle between DMI operations.

dmihardreset and **dmireset** are decoded from the DTMCS scan register (not from the DMI shift register). Writing **dmihardreset** clears the TAP-domain DMI soft state, including the selected DMI address, DATA registers, COMMAND, abstract-command error state, ABSTRACTAUTO state, and SBA address/data shadow registers. It does not replace the SoC system reset. **dmireset** is accepted as a no-op: this DTM keeps **dmistat** constant at 0 and never reports a sticky DMI operation error, so there is no per-operation status for **dmireset** to clear.

8.4. DMI Register Map

Unlisted DMI addresses read as zero and ignore writes. Optional authentication, configuration-string pointers, halt-on-reset requests, multi-hart arrays, extra program-buffer words, and SBA address/data words above 32 bits are absent. `dmstatus.authenticated` is fixed at one. The directed test scans every unimplemented DMI address for read-as-zero/write-ignore behavior.

The DMI register bank implements the following addresses.

Table 28. Implemented DMI Registers

Address	Register	Access	Implemented Behavior
0x04	DATA0	R/W	Primary abstract-command data register and command result register.
0x05	DATA1	R/W	Secondary abstract-command data register. Access-memory commands use it as the memory address.
0x10	DMCONTROL	R/W	Implements <code>dmactive</code> , <code>ndmreset</code> , <code>haltreq</code> , <code>resumereq</code> , <code>hartreset</code> , <code>ackhavereset</code> , and <code>hartsello</code> . While <code>dmactive</code> is 0 the Debug Module is held in reset: the abstract-command and SBA engines return to idle and every DM register (DATA0/DATA1, COMMAND, PROGBUF, ABSTRACTAUTO, <code>abstractcs.cmderr</code> , the SBCS fields, SBADDRESS0/SBDATA0, and the DMCONTROL control bits <code>haltreq/resumereq/hartreset/ndmreset/hartsel</code>) takes its reset value. <code>dmactive</code> is the only DMCONTROL bit that can be written while <code>dmactive</code> is 0; other field writes in that same DMI transaction are discarded. The DMI transport itself (the selected DMI address and the TCK↔system-clock dispatch handshake) is not part of the Debug Module and is unaffected, so ordinary DMI reads keep working across a <code>dmactive</code> 0→1 cycle. The shadowed hart CSRs (<code>dcsr/dpc/dscratch*</code>) and the trigger CSRs (<code>tselect/tdata1/tdata2</code>) are also not affected by <code>dmactive</code> ; they follow hart reset.

Address	Register	Access	Implemented Behavior
0x11	DMSTATUS	R/O	Reports single-hart halted/running/resume/nonexistent status, authenticated , {any,all}havereset , ndmresetpending , impebreak , and version = 3 (Debug Spec 1.0). hasresethaltreq , confstrptrvalid , and stickyunavail read 0.
0x12	HARTINFO	R/O	Returns the fixed RTL HARTINFO_VALUE .
0x16	ABSTRACTCS	R/W1C fields	Reports progbufsize=2 , datacount=2 , busy , and cmderr . Writing one to a cmderr bit clears the corresponding error state.
0x17	COMMAND	R/W	Dispatches a supported abstract command when the Debug Module is active and not busy.
0x18	ABSTRACTAUTO	R/W	Implements auto-execution controls for DATA0/DATA1 and PROGBUF0/PROGBUF1.
0x20	PROGBUF0	R/W	Program buffer word 0. Reset value is 0x0010_0073 (EBREAK) .
0x21	PROGBUF1	R/W	Program buffer word 1. Reset value is 0x0010_0073 (EBREAK) .
0x38	SBCS	R/W	System Bus Access (SBA) control/status. Implements 8-, 16-, and 32-bit access selection, read-on-address, read-on-data, autoincrement, busy, busy error, and SBA error reporting.
0x39	SBADDRESS0	R/W	32-bit System Bus Access (SBA) address.
0x3C	SBDATA0	R/W	32-bit System Bus Access (SBA) data register.
0x40	HALTSUM0	R/O	Bit 0 indicates that the implemented hart is halted.

The RTL contains one hart. A non-zero **hartsello** value selects a non-existent hart and is reflected through the DMSTATUS **anynonexistent/allnnonexistent** status. **hartsello** is only updated while no abstract command is in progress (**busy=0**); a write attempted during a command is ignored so hart selection stays stable for the running command.

dmstatus.anyhavereset/allhavereset reset to 1: any reset that reaches the Debug Module — power-on, the external reset pin, **ndmreset**, or **hartreset** — arms the bit, and it is cleared only by writing **ackhavereset**. It survives a **dmactive** 0 → 1 cycle.

8.5. Abstract Commands

The Debug Module implements Access Register and Access Memory command types.

8.5.1. Access Register

Supported register transfers use a 32-bit abstract access size.

The implemented register-number handling includes:

- Integer registers `x0–x31` through abstract register numbers `0x1000–0x101F`.
- `dpc` at CSR number `0x7B1`.
- Debug and machine CSRs below `0x1000` through the debug CSR path.
- `transfer=0`, `postexec=0` as a valid no-operation command.
- `postexec` execution through the two-word program buffer.

Commands requiring an unsupported access size report an abstract-command not-supported error (`cmderr=2`). Commands that require hart access while the hart is running report a halt/resume error (`cmderr=4`). An abstract access to a CSR the core does not implement fails with `cmderr=3` (exception) and performs no write — so a debugger probing for optional CSRs (e.g. vector `vlenb`, the AIA `mtopi/miselect` registers) is told the CSR is absent rather than receiving a spurious zero.

8.5.2. Access Memory

Access Memory supports 8-, 16-, and 32-bit transfers. `aamvirtual` is not implemented. Access sizes larger than 32 bits are reported as unsupported.

DATA1 supplies the address. DATA0 supplies or receives the transfer data. Address post-increment is supported by the RTL command engine.

8.6. System Bus Access (SBA)

SBA provides direct debug memory access independently of abstract Access Memory commands.

The RTL implements:

- 8-bit, 16-bit, and 32-bit accesses.
- `sbreadonaddr`.
- `sbreadondata`.

- **sbautoincrement**.
- **sbbusy** and **sbbusyclear**.
- SBA error reporting and write-one-to-clear error handling.

sbbusyclear is set whenever the debugger writes **SBADDRESS0** or **SBDATA0**, or reads **SBDATA0**, while a system bus access is in progress (**sbbusy=1**); the interfering **SBADDRESS0/SBDATA0** write is discarded (the shadow register keeps its value). No SBA access is started while **sberror** or **sbbusyclear** is set; both are cleared write-one-to-clear via **SBCS**.

The debug-bus response timeout (used by SBA and by abstract Access Memory) is a counter of parameterized width **DBG_BUS_TIMEOUT_W** (default 12 → 4095 cycles, about 41 μs at 100 MHz). A response later than the timeout reports **sberror=1** (SBA) or **abstractcs.cmderr=5** (abstract memory). Integrators with very slow debug targets can widen the parameter.

8.7. Debug CSRs

The Debug Module owns the debug CSRs listed below.

Table 29. Debug CSR Support

Address	CSR	Access	Behavior
0x7B0	dcsr	R/W, WARL/RO fields	Reports xdebugver=4 . prv is forced to M-mode (3). Implemented writable controls include ebreakm , stepie , stopcount , stoptime , mprven , and step . cause is hardware-written and read-only to the debugger.
0x7B1	dpc	R/W	Debug program counter. Reset value is 0x8000_0000 .
0x7B2	dscratch0	R/W	Debug scratch register 0.
0x7B3	dscratch1	R/W	Debug scratch register 1.

dcsr resets internally to **0x4000_0003**. Readback reconstructs read-only and WARL fields, including the hardware debug-entry cause.

8.8. Hardware Triggers

Trigger matching occurs in EX with timing-before behavior: a matching instruction is halted before it is promoted for normal execution. A trigger is eligible only when its type is 2, M-mode enable bit is set, and **action=1** requests Debug Mode entry.

The RTL supports:

- execute match against the instruction PC;
- load match against the effective data address;
- store match against the effective data address;
- exact address match (**match=0**);
- NAPOT address match (**match=1**) for load/store triggers.

Execute matching is exact-PC only in the current core. Unsupported **match** values fall back to exact comparison in the RTL helper.

If EBREAK debug-entry and a hardware trigger coincide, the debug-halt bookkeeping records the trigger as the halt cause (**trigger_halt**) and does not mark EBREAK as the separate halt cause. Multiple trigger slots may report hit simultaneously.

The default RTL implements two trigger slots (**N_TRIGGERS=2**). **N_TRIGGERS** is fully parameterized in both the core and the debug module: the trigger register bank, its reset values, the per-trigger hit-latch, and the **tselect** index are all generated from **N_TRIGGERS**, and the design elaborates for **N_TRIGGERS** = 1, 2, or 4. **tselect** is WARL to **[0, N_TRIGGERS)**.

Table 30. Trigger CSRs

Address	CSR	Behavior
0x7A0	tselect	Selects a trigger slot. Writes outside the implemented trigger count are ignored.
0x7A1	tdata1	Implements an mcontrol trigger. Every field is WARL-coerced on write to a value JV32 implements, so read-back reflects only supported behaviour: type=2 , action=1 (enter Debug Mode), match ∈ {0 exact, 1 NAPOT} (other values read back 0), and select/timing/sizelo/sizehi/chain/maskmax = 0. dmode , the M-mode enable, and execute/store/load are stored as written. The hardware hit state is reported in bit 20 and is write-zero-to-clear.
0x7A2	tdata2	Stores the trigger match value.
0x7A4	tinfo	Returns 0x0000_0004 , indicating support for trigger type 2.

The reset value of each implemented **tdata1** entry is **0x2000_0000**; **tdata2** resets to zero.

8.9. cJTAG Bridge

When `USE_CJTAG=1`, `cjtag_bridge.sv` converts the external two-wire TCKC/TMSC interface to the internal JTAG TAP signals.

The bridge implements:

- Two-stage synchronization of TCKC and TMSC into the system-clock domain.
- TCKC edge detection.
- TMSC escape-sequence detection.
- OScan1 serialized scan transport.
- TAP.7 star-2 activation behavior implemented by the state machine.
- Registered generation of the internal TAP clock and JTAG data/control signals.

The RTL requires each TCKC high and low phase to remain stable long enough to pass through the two-stage synchronizer and edge detector. The source comments specify the design requirement `f_sys >= 6 × f_tckc`. This is a functional sampling requirement, not an FPGA/ASIC timing-closure guarantee.

Chapter 9. Clock, Reset, and RTL Timing

9.1. Clock and Reset

9.1.1. Clock Domains

The SoC has two independent clock domains.

Table 31. Clock Domain Summary

Domain	Source	Nominal Frequency	Clocks
System (clk)	External pin	100 MHz (parameterized via CLK_FREQ)	jv32_core , TCM SRAM, AXI crossbar, UART, CLIC, Magic, debug system clock path
TAP (tck / tap_tck)	External TCK (USE_CJTAG=0) or bridge-generated TAP clock (USE_CJTAG=1)	Integration-dependent	jtag_tap and the TAP-domain DMI register bank. In cJTAG mode, the bridge samples TCKC using clk .

The TAP and system-clock domains are asynchronous in 4-wire JTAG mode. CDC handling is split between **jtag_tap** and the system-clock debug engine: toggle/stable-payload handshakes and synchronizer stages are used at the boundary. **jv32_dtm** itself is a system-clock-domain module; it is not the sole location of the CDC synchronizers.

9.1.2. System Clock (**clk**)

The system clock drives the processor pipeline, TCM, peripherals, and the system-clock half of the debug transport module.

Table 32. System Clock Specifications

Parameter	Value	Description
Pin	clk	Single-ended clock input
Nominal frequency	100 MHz RTL default	Parameterized via CLK_FREQ ; affects UART baud divisor computation. The RTL default is a functional configuration value, not a defined implementation frequency.

Parameter	Value	Description
Minimum frequency	No lower limit specified	Must satisfy $\text{CLK_FREQ} / \text{BAUD_RATE} \rightarrow 4$ for correct UART operation
Maximum frequency	Technology-dependent	Not defined by the RTL. Use synthesis, static timing analysis, and implementation results for the selected FPGA or ASIC target.

9.1.3. TAP Clock — **USE_CJTAG=0** (4-wire JTAG)

In 4-wire JTAG mode, `jtag_pin0_tck_i` directly clocks the TAP. TCK is asynchronous to the system clock. Status and command information crossing between the TAP and system-clock domains is synchronized in the debug logic.

The RTL does not define a universal maximum external TCK frequency. The usable TCK frequency depends on the implementation timing and on the clock-domain-crossing handshake latency. The DTMCs `idle=1` field requires the debugger to insert at least one Run-Test/Idle TCK cycle between DMI operations.

9.1.4. TAP Clock — **USE_CJTAG=1** (2-wire cJTAG)

In 2-wire cJTAG mode, `jtag_pin0_tck_i` carries TCKC and `jtag_pin1_tms_i/o` carries TMSC. `cjtag_bridge` samples both external signals using the system clock, detects synchronized TCKC edges, and generates the internal TAP clock and JTAG scan signals.

The bridge source defines the functional sampling requirement:

$$f_{\text{sys}} \geq 6 \times f_{\text{tckc}}$$

This corresponds to at least three system-clock cycles for each TCKC high or low phase, allowing the two-stage synchronizer and edge detector to observe the signal reliably at the RTL interface level.

The ratio above is a functional design requirement. Actual board-level and FPGA/ASIC timing margins must be established by the target implementation.

9.1.5. Reset Architecture

The SoC implements an **asynchronous-assert** / **synchronous-deassert** reset pattern. Two independent sources can assert reset: the external `rst_n` pin and the debug non-debug-module reset (`dbg_ndmreset`) asserted by the RISC-V Debug Module via JTAG.

9.1.5.1. Reset signal flow

```
// jv32_soc.sv
assign rst_n_pre = rst_n & ~dbg_ndmreset;    // (1) OR of both reset sources

always_ff @(posedge clk or negedge rst_n_pre) begin
    if (!rst_n_pre) begin
        rst_sync_ff1 <= 1'b0;                // (2) async assert: both FFs cleared
    immediately
        rst_sync_ff2 <= 1'b0;
    end else begin
        rst_sync_ff1 <= 1'b1;                // (3) synchronous deassert: propagates over 2
    clk cycles
        rst_sync_ff2 <= rst_sync_ff1;
    end
end

assign soc_rst_n = rst_sync_ff2;             // (4) clean synchronised reset to all SoC
logic
```

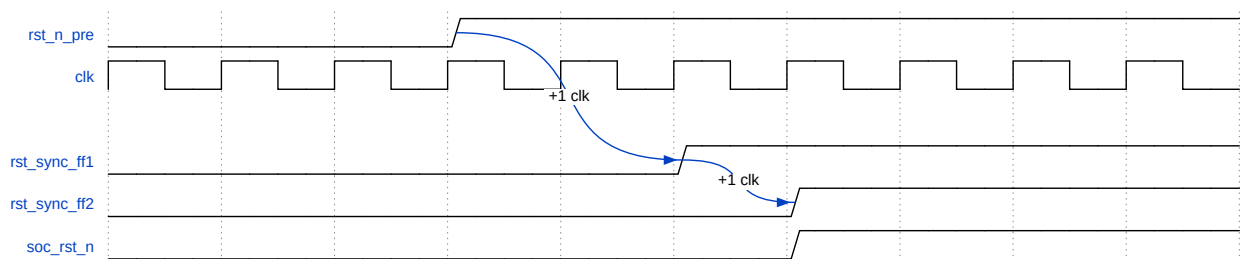


Figure 1. Reset de-assertion timing (2-cycle synchroniser)

9.1.5.2. Reset source behavior

Table 33. Reset Sources

Source	Assertion	Description
rst_n	Asynchronous (active-low)	External system reset pin; asserted immediately by hardware; de-asserted synchronously through the 2-FF chain
dbg_ndmreset	Asynchronous (active-high) from jv32_dtm	Non-debug-module reset issued over JTAG (dmcontrol.ndmreset); resets the SoC while the debug module itself remains active; rst_n_pre = rst_n & ~dbg_ndmreset
dbg_hartreset	Core-local reset request	dbg_hartreset is a TAP-domain level passed through the debug engine and combined into core_rst_n = rst_n & ~dbg_hartreset_i . It resets the core without resetting the peripheral subsystem or TAP. Unlike ndmreset , its release does not pass through the SoC two-flop reset synchronizer. Reset recovery/removal timing must be checked during physical integration.

9.1.5.3. Reset domain coverage

`soc_rst_n` (the synchronised output of the 2-FF chain) is distributed to:

- `jv32_top` — processor pipeline, TCM SRAM control registers, AXI master/slave
- `axi_clic` — CLIC/CLINT registers (`mtime`, `mtimecmp`, `msip`, interrupt pending)
- `axi_uart` — UART TX/RX FIFOs and baud rate generator
- `axi_xbar` — AXI crossbar state

The JTAG TAP (`jtag_tap`) and `jv32_dtm` are reset by `rst_n` **directly** (not through the synchroniser), so debug infrastructure is available as soon as the external reset is released, independent of the 2-cycle delay on `soc_rst_n`. `jtag_ntrst_i` provides an additional TAP-domain reset. System reset now clears both the TAP dispatch toggles and their receiving engine, preventing completed commands from replaying after reset. `ndmreset` leaves both debug domains active.

9.1.5.4. Reset timing requirements

Table 34. Reset Timing Requirements

Parameter	Minimum	Description
<code>rst_n</code> assertion	Asynchronous	The RTL asynchronously clears the reset synchronizer when <code>rst_n_pre</code> is low. Minimum external reset pulse width is an implementation/electrical requirement and is not specified by the RTL.
<code>rst_n</code> deassert hold (before first <code>clk</code> rising edge used for computation)	2 clock cycles	Two <code>clk</code> rising edges are required before <code>soc_rst_n</code> deasserts and SoC logic begins operating
<code>rst_n</code> setup before <code>clk</code> rising edge	Not required	Deassertion is sampled synchronously; no setup requirement on the deassert edge
<code>dbg_ndmreset</code> assertion	Clock-domain generated	The signal is a TAP-domain register level passed through the debug engine to the asynchronous assertion path of the SoC reset synchronizer. No independent external pulse-width requirement is defined by this RTL interface.



The `rst_n_pre` net is a 2-input LUT driving the asynchronous clear (`CLR`) of `rst_sync_ff1` and `rst_sync_ff2`. In FPGA implementations this is an

intentional asynchronous-reset pattern; the path is marked as a false path to suppress the Vivado LUTAR-1 methodology warning (`fpga/scripts/constraints.xdc`).

9.2. Timing Parameters

The RTL does not define FPGA/ASIC setup time, hold time, clock-to-output time, or maximum operating frequency. Any numerical implementation timing values must come from a named synthesis/place-and-route target and timing report. Values in this section therefore describe protocol or RTL-cycle requirements only.

Table 35. Timing Characteristics (100 MHz, $T_{clk} = 10\text{ ns}$)

Parameter	Min	Max	Description
Setup time (AXI inputs)	2.0 ns	—	Input setup before clock edge
Hold time (AXI inputs)	1.0 ns	—	Input hold after clock edge
Clock-to-output (AXI outputs)	—	8.0 ns	Output valid after clock edge
TCM read latency	1 cycle	—	Registered SRAM output
Load-to-use latency	2 cycles	—	1 EX cycle + 1 stall cycle
Branch flush penalty	0 / 1 cycle	—	0 cycles on correct BTFNT prediction; 1 cycle on misprediction or when <code>BP_EN=0</code> and branch is taken

9.3. Top-Level Parameters

Table 36. `juv32_soc` Configurable Parameters

Parameter	Default	Description
<code>CLK_FREQ</code>	<code>100_000_000</code>	System clock frequency in Hz; used to compute UART baud divisor
<code>BAUD_RATE</code>	<code>115_200</code>	UART baud rate (must satisfy <code>CLK_FREQ / BAUD_RATE >= 4</code>)
<code>UART_FIFO_DEPTH</code>	<code>16</code>	TX and RX FIFO depth (same value for both; must be a power-of-2)

Parameter	Default	Description
RV32E_EN	0	1=RV32E reduced register file (x0–x15); 0=RV32I register file (x0–x31). In RV32E mode, the decoder traps an instruction as illegal when an actually-used rs1 , rs2 , or written rd selects x16–x31. RV32E mode also disables the Zba/Zbb/Zbs datapath, RAS, and instruction prefetch buffer in rv32_core , and changes the mis a base-ISA indication.
RV32M_EN	1	1=M-extension enabled (MUL/DIV/REM instructions legal); 0=all multiply/divide instructions trap as illegal
JTAG_EN	1	1=JTAG debug port and RISC-V Debug Module instantiated; 0=all debug outputs tied off, JTAG pins ignored
USE_CJTAG	0	0=4-wire JTAG (TCK/TMS/TDI/TDO); 1=2-wire cJTAG IEEE 1149.7 OScan1 (TCKC/TMSC); see Clock and Reset for clock architecture
JTAG_IDCODE	32'h1DEAD3FF	JTAG TAP IDCODE register value returned by the DTM on IR= IDCODE (0x01) scan
N_TRIGGERS	2	Number of hardware breakpoint/watchpoint trigger registers (tdata1 / tdata2 pairs); verified for 1, 2, and 4
DBG_BUS_TIMEOUT_W	12	Width of the debug-bus (SBA and abstract Access Memory) response-timeout counter. Default 12 → 4095 cycles (~41 μs at 100 MHz). Increase for slower external debug targets.
AMO_EN	1	1=A-extension enabled (LR.W/SC.W and AMO{SWAP,ADD,XOR,AND,OR,MIN,MAX,MINU,MAXU}.W); 0=all atomic instructions using the AMO opcode, including LR.W/SC.W, trap as illegal. Verified by sw/amo_config and testbench/compliance/tb_disabled_isa.sv .
FAST_MUL	1	1=fast multiplier selected (mode set by MUL_MC); 0=multi-cycle serial shift-and-add multiply (CLZ-reduced; 1–32 cycles)
MUL_MC	1	When FAST_MUL =1: 1=2-stage pipelined multiplier (2-cycle result), 0=1-cycle combinatorial multiplier. Ignored when FAST_MUL =0.

Parameter	Default	Description
SCOREBOARD_EN	1	1=enable single-slot non-blocking scoreboard for multi-cycle MUL with ordered WB retirement. Blocks issue on RAW/WAW/structural/commit-sensitive hazards. 0=disable scoreboard (all multi-cycle ALU ops use blocking path). In jv32_top, it is auto-gated off when FAST_MUL=1, MUL_MC=0, FAST_DIV=1, and FAST_SHIFT=1 (all arithmetic single-cycle).
FAST_DIV	0	1=single-cycle combinatorial divide; 0=CLZ-reduced serial divide (1–32 cycles)
FAST_SHIFT	1	1=single-cycle barrel shift; 0=iterative shift (up to 31 cycles)
BP_EN	1	1=BTENT static branch predictor enabled (backward branches predicted taken, forward branches predicted not-taken); 0=all branches predicted not-taken. EX-stage correction behavior is described in the branch-prediction section.
RAS_EN	1	1=2-entry Return Address Stack enabled for eligible return prediction; 0=return prediction through the RAS is disabled. The RAS is automatically disabled when RV32E_EN=1.
IBUF_EN	1	1=2-entry instruction prefetch buffer enabled; 0=disabled. The buffer is automatically disabled when RV32E_EN=1.
ZCMP_EN	1	1=Zcmp extension enabled (cm.push, cm.pop, cm.popret, cm.popretz, cm.mvsa01, cm.mva01s); 0=these instructions trap as illegal.
RV32B_EN	1	1=Zba/Zbb/Zbs B-extension enabled (SH1ADD/SH2ADD/SH3ADD, bit-manipulation, etc.); 0=all B-extension instructions trap as illegal (synthesized away).
IRAM_SIZE	131072	IRAM size in bytes (power-of-2; also sets TCM address decode mask)
DRAM_SIZE	131072	DRAM size in bytes (power-of-2)
BOOT_ADDR	32'h8000_0000	Reset PC (word-aligned). Must equal IRAM_BASE for the core to boot from IRAM.
IRAM_BASE	32'h8000_0000	IRAM TCM base address and address-decode base. Normally equals BOOT_ADDR.
DRAM_BASE	32'h9000_0000	DRAM base address and TCM address decode start

Chapter 10. Programming Guide

10.1. MMIO Access Rules

Peripheral registers are 32-bit memory-mapped registers unless a register description states otherwise. Software should use naturally aligned 32-bit accesses for control and status registers. Reserved offsets must not be used as part of the programming interface.

For device programming sequences, software should complete required configuration writes before enabling the corresponding interrupt or operation. The current RTL treats **FENCE** as a no-op, so software must not rely on it to create an additional hardware data-memory barrier. **FENCE.I** is implemented separately and flushes/refetches the instruction path for instruction-memory synchronization.

10.2. Access and Error Handling

Software must treat an AXI error response as a failed access. Error behavior is target-specific: UART and Magic can return **SLVERR** for unsupported addresses, while CLIC returns **OKAY** and zero/no-effect behavior for unsupported offsets. Addresses routed to the external AXI interface may return any response permitted by the connected target or interconnect.

Software and external masters should use naturally aligned accesses. Several RTL blocks select registers or SRAM words using upper address bits and do not independently reject all misaligned accesses.

10.3. Boot Sequence

The processor boots from **BOOT_ADDR** (default **0x8000_0000** = IRAM base). The reset vector is hardwired to **BOOT_ADDR & ~3** (word-aligned).

For simulation, the typical boot flow is:

1. ELF loaded into TCM via DPI (**mem_write_byte**) before simulation starts
2. Simulation starts, core fetches from **BOOT_ADDR** (mapped to IRAM, 1-cycle latency)
3. Startup code (**startup.S**) initialises stack pointer to top of DRAM
4. Hardware CSRs initialized: **mstatus**, **mtvec**, **mie**
5. Peripherals initialized (UART starts with the parameter-derived baud divisor; software may change it at runtime; CLIC **mtimecmp** is programmed as required)

6. `mstatus.MIE` set to enable interrupts
7. Jump to `main()`

10.4. Stack Configuration

Default linker script maps DRAM to data/BSS/stack:

```
Stack top: DRAM_BASE + DRAM_SIZE - 4    (e.g. 0x9001_FFFC)
Stack grows downward into DRAM
Data/BSS: DRAM_BASE (loaded by startup.S from IRAM)
```

10.5. Interrupt Handling

10.5.1. Setting Up Interrupts

```
#include "csr.h"

extern void trap_entry(void); // defined in startup.S

void setup_interrupts(void) {
    // Set trap vector (direct mode: mtvec points to handler)
    write_csr(mtvec, (uint32_t)trap_entry);

    // Enable M-mode interrupts in mstatus
    set_csr(mstatus, 0x8); // MIE bit

    // Enable timer + external + software interrupts in mie
    set_csr(mie, (1<<7) | (1<<11) | (1<<3)); // MTIE | MEIE | MSIE

    // Program CLIC mtimecmp for periodic 1ms timer (100 MHz)
    volatile uint32_t *mtime_lo    = (uint32_t *)0x02004000;
    volatile uint32_t *mtime_hi    = (uint32_t *)0x02004004;
    volatile uint32_t *mtimecmp_lo = (uint32_t *)0x02004008;
    volatile uint32_t *mtimecmp_hi = (uint32_t *)0x0200400C;

    // Set mtimecmp_hi first to avoid spurious IRQ
    *mtimecmp_hi = 0xFFFFFFFF;
    uint64_t now = ((uint64_t)*mtime_hi << 32) | *mtime_lo;
    uint64_t cmp = now + 100000; // 1 ms @ 100 MHz
    *mtimecmp_lo = (uint32_t)cmp;
    *mtimecmp_hi = (uint32_t)(cmp >> 32);
}
```

10.5.2. Trap Handler

```
void trap_handler(uint32_t mcause_val) {
    if (mcause_val & 0x80000000) {
        // Interrupt
        uint32_t cause = mcause_val & 0x7FFFFFFF;
```

```

    switch (cause) {
    case 7: // Machine timer interrupt
        handle_timer_irq();
        break;
    case 11: // Machine external interrupt (CLIC)
        handle_clic_irq();
        break;
    case 3: // Machine software interrupt
        handle_sw_irq();
        break;
    }
} else {
    // Synchronous exception
    handle_exception(mcause_val);
}
}

void handle_timer_irq(void) {
    // Clear by advancing mtimecmp
    volatile uint32_t *cmp_lo = (uint32_t *)0x02004008;
    volatile uint32_t *cmp_hi = (uint32_t *)0x0200400C;
    uint64_t cmp = ((uint64_t)*cmp_hi << 32) | *cmp_lo;
    cmp += 100000; // next 1 ms
    *cmp_hi = 0xFFFFFFFF; // prevent spurious IRQ during update
    *cmp_lo = (uint32_t)cmp;
    *cmp_hi = (uint32_t)(cmp >> 32);
}

void handle_clic_irq(void) {
    // CLIC vectored entry: mcause = 0x8000_000B, mintstatus.mil = this IRQ's level.
    // Scan CLICINT[n].ip to identify the source; IP auto-clears when ext_irq_i[n]
    deasserts.
    for (int n = 0; n < 16; n++) {
        volatile uint32_t *clicint = (uint32_t *) (0x02001000 + n * 4);
        if (*clicint & 0x1) {
            // Source n is pending – handle it here.
        }
    }
    // Software-assisted tail-chain: check for next pending IRQ via mnxti.
    // If a higher-priority IRQ arrived during this handler, chain directly to it.
    uintptr_t next = read_csr(mnxti);
    if (next) {
        // Claim it (sets mcause, mintstatus.mil, re-enables MIE) and branch.
        write_csr(mnxti, 0);
        ((void (*)(void))next)();
    }
    // Hardware tail-chain: if another CLIC IRQ above mintthresh is pending
    // when this handler's mret executes, the CPU redirects to mtvt+4*id
    // automatically – no code change needed.
}

```

10.6. UART Usage

```

#define UART_DATA    (*(volatile uint32_t *)0x20010000)
#define UART_STATUS  (*(volatile uint32_t *)0x20010004)
#define UART_IE      (*(volatile uint32_t *)0x20010008)

```

```
#define UART_IS      (*(volatile uint32_t *)0x2001000C)

// Poll-mode transmit
void uart_putc(char c) {
    while (UART_STATUS & 0x1); // wait until TX not full
    UART_DATA = c;
}

// Poll-mode receive
char uart_getc(void) {
    while (!(UART_STATUS & 0x4)); // wait until RX not empty
    return (char)(UART_DATA & 0xFF);
}

// Print string
void uart_puts(const char *s) {
    while (*s) uart_putc(*s++);
}
```

10.7. Simulation Debugging

10.7.1. Magic Device Console

```
#define MAGIC_CON (*(volatile uint32_t *)0x40000000)
#define MAGIC_EXIT (*(volatile uint32_t *)0x40000004)

void debug_putc(char c) { MAGIC_CON = c; }
void debug_puts(const char *s) { while (*s) debug_putc(*s++); }
void sim_pass(void) { MAGIC_EXIT = 1; } // HTIF: exit(0)
void sim_fail(int code) { MAGIC_EXIT = (code << 1) | 1; } // HTIF: exit(code)
```

10.7.2. Waveform Capture

Generate VCD waveform for GTKWave analysis:

```
make WAVE=1 rtl-hello
gtkwave build/tb_jv32_soc.vcd
```

Key signals to observe:

- **Pipeline:** `jv32_core.pc_if`, `jv32_core.pc_ex`, `jv32_core.pc_wb`
- **TCM:** `jv32_top.imem_req_valid`, `jv32_top.imem_resp_valid`,
`jv32_top.dmem_req_valid`
- **AXI master:** `jv32_top.m_axi_arvalid`, `jv32_top.m_axi_rvalid`
- **Interrupts:** `jv32_soc.timer_irq`, `jv32_soc.clic_irq`
- **Heartbeat:** `jv32_soc.heartbeat_o` — toggles every 2^{24} retired instructions; a slow square

wave indicating the core is making forward progress

10.7.3. Instruction Trace

Software and RTL traces can be compared for regression:

```
make compare-hello
diff sim_trace.txt rtl_trace.txt
```

Trace format: <cycle> <pc> <hex_instruction> <disassembly>

10.8. Performance-Oriented Programming Considerations

The following items are implementation-aware guidance, not architectural requirements:

- Place latency-sensitive code and data in TCM when deterministic local-memory access is desired.
- Schedule independent instructions between a load and its first consumer when practical to avoid a load-use stall.
- Select **FAST_MUL**, **FAST_DIV**, and **FAST_SHIFT** according to the required area/latency trade-off.
- Avoid unnecessary D-side accesses to IRAM when instruction-fetch contention matters.
- Measure complete workloads on the target configuration; this datasheet does not define a typical CPI or system performance target. <<<

Chapter 11. Register Reference Index

The following block sections contain the register definitions. Use these definitions for access rules, side effects, parameter dependencies, and reset behavior:

- [Machine CSRs](#)
- [Local interrupt controller and timer registers](#)
- [UART registers](#)
- [Magic simulation registers](#)
- [Debug Module registers](#)

Appendix A: Trap Cause Reference

This section summarizes trap cause encodings implemented or recognized by the current RTL. See [Exception and Interrupt Handling](#) for behavior and trap-entry sequencing.

Table 37. Implemented Machine-Mode Trap Causes (*mcause* encoding)

Code	Type	Description	Source
1	Exception	Instruction access fault	I-fetch AXI <i>SLVERR</i> / <i>DECERR</i>
2	Exception	Illegal instruction	Core decode
3	Exception	Breakpoint	EBREAK
4	Exception	Load address misaligned	Core; normal loads and misaligned LR.W
5	Exception	Load access fault	D-bus AXI <i>SLVERR</i> / <i>DECERR</i>
6	Exception	Store address misaligned	Core; normal stores and misaligned SC.W / AMO.W
7	Exception	Store/AMO access fault	D-bus AXI <i>SLVERR</i> / <i>DECERR</i>
11	Exception	ECALL from M-mode	ECALL instruction
<i>0x8000_0003</i>	Interrupt	Machine software interrupt	CLIC MSIP[0]
<i>0x8000_0007</i>	Interrupt	Machine timer interrupt	CLIC <i>mtime</i> >= <i>mtimecmp</i>
<i>0x8000_000B</i>	Interrupt	Machine external interrupt	CLIC any <i>CLICINT[n].ie</i> & <i>ip</i> with level > <i>mintthresh</i>



mcause[31]=1 indicates interrupt; *mcause*[30:0] is the cause code.

Appendix B: Pin Descriptions

This section is the integration-facing signal reference. Signal behavior is defined in the corresponding functional sections; this section provides direction, width, and interface-level purpose.

B.1. Clock and Reset Pins

Table 38. Clock and Reset Pins

Signal	I/O	Width	Description
<code>clk</code>	Input	1	System clock (100 MHz nominal)
<code>rst_n</code>	Input	1	Asynchronous active-low reset signal

B.2. UART Interface

Table 39. UART Pins

Signal	I/O	Width	Description
<code>uart_tx_o</code>	Output	1	UART transmit data
<code>uart_rx_i</code>	Input	1	UART receive data (asynchronous)

B.3. External Interrupt Interface

Table 40. External Interrupt Pins

Signal	I/O	Width	Description
<code>ext_irq_i[15:0]</code>	Input	16	External interrupt request lines, routed to CLIC <code>CLICINT[n].ip</code> ; active-high, level-sensitive

B.4. TCM AXI Slave Interface

Table 41. TCM Slave Port Pins (exposed as `s_tcm_*` on `juv32_soc`)

Signal	I/O	Width	Description
<code>s_tcm_araddr[31:0]</code>	Input	32	Read address
<code>s_tcm_arvalid</code>	Input	1	Read address valid

Signal	I/O	Width	Description
s_tcm_arready	Output	1	Read address ready
s_tcm_rdata[31:0]	Output	32	Read data
s_tcm_rresp[1:0]	Output	2	Read response
s_tcm_rvalid	Output	1	Read data valid
s_tcm_rready	Input	1	Read data ready
s_tcm_awaddr[31:0]	Input	32	Write address
s_tcm_awvalid	Input	1	Write address valid
s_tcm_awready	Output	1	Write address ready
s_tcm_wdata[31:0]	Input	32	Write data
s_tcm_wstrb[3:0]	Input	4	Write byte strobes
s_tcm_wvalid	Input	1	Write data valid
s_tcm_wready	Output	1	Write data ready
s_tcm_bresp[1:0]	Output	2	Write response
s_tcm_bvalid	Output	1	Write response valid
s_tcm_bready	Input	1	Write response ready

B.5. JTAG / cJTAG Debug Interface

Active only when **JTAG_EN=1**. Pin roles differ between **USE_CJTAG=0** (4-wire JTAG) and **USE_CJTAG=1** (2-wire cJTAG).

Table 42. JTAG / cJTAG Debug Pins

Signal	I/O	Width	Description
<code>jtag_ntrst_i</code>	Input	1	TAP reset (active-low); resets the JTAG TAP state machine independently of <code>rst_n</code> . Tie high if not used.
<code>jtag_pin0_tck_i</code>	Input	1	<code>USE_CJTAG=0</code> : TAP clock (TCK), driven by debug probe, asynchronous to <code>clk</code> . <code>USE_CJTAG=1</code> : cJTAG serial clock (TCKC); asynchronous to <code>clk</code> ; internally sampled by 2-stage synchroniser in <code>cjtag_bridge</code> . Maximum frequency = $f_{sys}/6$.
<code>jtag_pin1_tms_i</code>	Input	1	<code>USE_CJTAG=0</code> : TAP Mode Select (TMS). <code>USE_CJTAG=1</code> : cJTAG data (TMSC) input direction; driven by probe during scan-in.
<code>jtag_pin1_tms_o</code>	Output	1	<code>USE_CJTAG=0</code> : not driven (tie <code>jtag_pin1_tms_oe</code> low). <code>USE_CJTAG=1</code> : TMSC output direction; valid when <code>jtag_pin1_tms_oe=1</code> (SoC driving TMSC during scan-out).
<code>jtag_pin1_tms_oe</code>	Output	1	TMSC output-enable (active-high). Drive the <code>OE</code> of an external IOBUF on TMSC. In <code>USE_CJTAG=0</code> mode this is always 0.
<code>jtag_pin2_tdi_i</code>	Input	1	<code>USE_CJTAG=0</code> : TAP Data In (TDI). <code>USE_CJTAG=1</code> : not used (internal cJTAG shift path is fully serialised over TMSC).
<code>jtag_pin3_tdo_o</code>	Output	1	<code>USE_CJTAG=0</code> : TAP Data Out (TDO); valid on falling edge of TCK. <code>USE_CJTAG=1</code> : not used; cJTAG scan-out uses TMSC.
<code>jtag_pin3_tdo_oe</code>	Output	1	TDO output-enable (active-high). Drive the <code>OE</code> of an external tristate buffer on TDO. In <code>USE_CJTAG=0</code> mode this follows the TAP shift state.

B.6. Trace Interface

Present in simulation only (excluded from synthesis via `ifndef SYNTHESIS`). All outputs change on the rising edge of `clk` and are qualified by `trace_valid`. Connect to the testbench trace logger.

Table 43. Trace Output Pins

Signal	I/O	Width	Description
<code>trace_en</code>	Input	1	Run-time trace enable gate; when low, <code>trace_valid</code> stays deasserted
<code>trace_valid</code>	Output	1	Strobe: high for one <code>clk</code> cycle when a new instruction has retired
<code>trace_reg_we</code>	Output	1	Register file write enable for the retired instruction
<code>trace_pc</code>	Output	32	PC of the retired instruction
<code>trace_rd</code>	Output	5	Destination register index of the retired instruction
<code>trace_rd_data</code>	Output	32	Value written to <code>rd</code> (valid when <code>trace_reg_we=1</code>)
<code>trace_instr</code>	Output	32	32-bit expanded instruction word (after RVC expansion)
<code>trace_mem_we</code>	Output	1	Memory write-enable for the retired instruction (store)
<code>trace_mem_re</code>	Output	1	Memory read-enable for the retired instruction (load)
<code>trace_mem_addr</code>	Output	32	Memory address of the load/store
<code>trace_mem_data</code>	Output	32	Data written to memory (store) or loaded from memory (load)
<code>trace_irq_taken</code>	Output	1	Strobe: high for one cycle when an interrupt is taken (pipeline redirected to handler)
<code>trace_irq_cause</code>	Output	32	<code>mcause</code> value captured at interrupt entry
<code>trace_irq_epc</code>	Output	32	<code>mepc</code> value captured at interrupt entry (pre-empted PC)
<code>trace_irq_store_we</code>	Output	1	Interrupt pipeline — memory write-enable for interrupt-triggered store (e.g. stack save)
<code>trace_irq_store_addr</code>	Output	32	Interrupt pipeline — store address

Signal	I/O	Width	Description
trace_irq_store_data	Output	32	Interrupt pipeline — store data

B.7. Branch Predictor Observability (**perf_bp_***)

One-cycle pulse outputs that count branch-predictor events. Active only with **BP_EN=1**; tied to 0 otherwise. Intended for performance counters in a testbench or external logic analyzer.

Table 44. Branch Predictor Observability Pins

Signal	I/O	Width	Description
perf_bp_branch	Output	1	A conditional branch retired this cycle
perf_bp_taken	Output	1	The conditional branch was taken
perf_bp_mispred	Output	1	Branch prediction was wrong (EX-stage redirect)
perf_bp_jal	Output	1	A JAL instruction retired this cycle
perf_bp_jal_miss	Output	1	JAL was not pre-decoded (caused an EX-stage redirect)
perf_bp_jalr	Output	1	A JALR instruction retired this cycle (is resolved in EX; eligible returns may already have a RAS prediction)

B.8. Heartbeat Output

heartbeat_o is a single-bit observability output derived from bit 24 of the **minstret** counter in **fv32_csr**. It changes state as instructions retire and can be connected to an LED, logic analyzer, or testbench monitor.

Table 45. Heartbeat Pin

Signal	I/O	Width	Description
heartbeat_o	Output	1	minstret[24] . The output toggles every 2^{24} retired instructions. One complete low-to-high-to-low period spans 2×2^{24} retired instructions.

The period depends on the average cycles per retired instruction (CPI):

$$T = (2 \times 2^{24} \times \text{CPI_avg}) / f_{\text{clk}}$$

For example, at the default 100 MHz system clock and **CPI_avg** = 1.0, one complete period is approximately 336 ms. Real software may have a longer period because stalls, memory accesses, branches, and multi-cycle operations increase CPI.

B.9. External AXI Master Interface (**ext_axi_***)

The external AXI master port carries transactions that fall outside all TCM and peripheral apertures (crossbar slave 3 catch-all). Connect to an external memory or MMIO fabric, or tie **ext_axi_arready**, **ext_axi_rvalid**, **ext_axi_awready**, **ext_axi_wready**, **ext_axi_bvalid** all to 0 if no external master is needed (the core will stall until reset).

Table 46. External AXI Master Pins

Signal	I/O	Width	Description
ext_axi_araddr[31:0]	Output	32	Read address
ext_axi_arvalid	Output	1	Read address valid
ext_axi_arready	Input	1	Read address ready
ext_axi_rdata[31:0]	Input	32	Read data
ext_axi_rresp[1:0]	Input	2	Read response (OKAY / SLVERR / DECERR)
ext_axi_rvalid	Input	1	Read data valid
ext_axi_rready	Output	1	Read data ready
ext_axi_awaddr[31:0]	Output	32	Write address
ext_axi_awvalid	Output	1	Write address valid
ext_axi_awready	Input	1	Write address ready
ext_axi_wdata[31:0]	Output	32	Write data

Signal	I/O	Width	Description
ext_axi_wst rb[3:0]	Output	4	Write byte strobes
ext_axi_wva lid	Output	1	Write data valid
ext_axi_wre ady	Input	1	Write data ready
ext_axi_bre sp[1:0]	Input	2	Write response
ext_axi_bva lid	Input	1	Write response valid
ext_axi_bre ady	Output	1	Write response ready

Chapter 12. Release and Integration Checklist

Before releasing an RTL configuration or software package, verify:

- ISA feature parameters and the resulting programmer-visible instruction set.
- `misa` against the enabled MISA-visible extensions.
- IRAM/DRAM sizes, linker-script regions, and canonical/alias address use.
- UART clock assumptions and the reset/default baud divisor.
- CLIC interrupt count, vector base, `mintthresh`, and enabled interrupt controls.
- External AXI address ownership and expected responses for non-internal addresses.
- JTAG/cJTAG mode selection, TAP connectivity, debug reset behavior, and supported DMI flow.
- SRAM/TCM initialization mechanism for the selected simulation, FPGA, or ASIC environment.
- Software headers and register definitions against the detailed block sections in this datasheet.
- Verification results for ISA compliance, privileged-architecture behavior, debug interoperability, AXI protocol compliance, target timing closure, and target-specific electrical requirements.

Any parameter change that affects software-visible behavior should update the RTL, this datasheet, software headers, linker configuration, and verification environment together.

Chapter 13. Verification and Compliance Status

Measured on 2026-09-17. Commands, configuration boundaries, and remaining requirements are recorded in [verif/COMPLIANCE_RESULTS.md](#). The final regression passed all 42 commands. A passing test establishes the tested behavior, not complete specification conformance.

Area	Implementation	Verification	Compliance claim
Base ISA and extensions	Parameterized RV32I/E with optional M/A/B/Zcmp groups	24 effective ISA configurations passed 2,120 applicable architectural-test executions; Zcmp has separate project-model and directed coverage	Configured tests verified; not independent certification of every extension
CSRs, traps, interrupts	Machine-mode subset and project-specific CLIC-like sideband	82 cycle-level CSR checks and CPU-generated fault checks verify exact cause, PC, and trap value; trap/nested-interrupt regressions passed	Tested behavior verified; no full privileged-architecture claim
Local interrupt controller and timer	CLINT-style registers and CLIC-like external control	100 randomized transactions and AXI response stability assertions passed (1,176 checks at the default seed)	No full CLIC or CLINT claim
AXI and memory	Implemented slave/master paths and TCM aliases	RAM, UART, Magic, CLIC, both TCM slave ports, and crossbar tests; all-channel external-memory stalls; 34 address-boundary probes	Implemented single-beat paths tested with stability assertions; not AXI certification
JTAG and Debug Module	RISC-V Debug Specification 1.0-oriented subset	Directed checks include unsupported DMI/IR values, independent resets, slow/fast/jittered TCK; OpenOCD and GDB interoperability tested	Tested interoperability verified; no complete Debug Spec conformance claim

Reset and CDC	Distributed synchronizers and debug transfer protocols	Two-edge SoC reset release, TCM retention, independent debug resets, and bundled-payload assertions tested; crossing inventory in verif/CDC_REVIEW.md	Simulation evidence only; exhaustive reset-phase and structural CDC signoff remain open
---------------	--	---	---

Chapter 14. Revision History

Table 47. Revision History

Revision	Date	Summary
1.4	2026-09-17	Fixed AXI response retention, independent UART write channels and strobes, TCM concurrent-request arbitration, crossbar duplicate reads, and debug command replay after system reset. Added the 24-configuration ISA matrix, address-boundary tests, reset checks, and CDC inventory. Added directed compliance regressions and measured verification status. Corrected MSIP byte strobes, zero-strobe MTIME writes, and AMO parameter documentation. Reviewed against the supplied RTL and reorganized the SoC datasheet to match the current RTL. Clarified ISA and CSR behavior, memory map and TCM aliases, CLIC, UART, AXI, debug/JTAG, atomic operations, exceptions, control-flow behavior, and integration requirements.

Chapter 15. References

15.1. Verification Boundary

This document is intended to match the supplied RTL source set. Functional correctness, ISA compliance, AXI compliance, timing closure, and target-specific electrical behavior require their respective verification and implementation flows. The datasheet does not replace those verification results.

The RTL is the implementation source of truth for this datasheet. When modifying programmer-visible behavior, update the RTL and this document in the same change set.

Within this document, each detailed block section is authoritative for that block's behavior. The register index links to these definitions, and appendices are summaries only. If a summary omits a restriction, side effect, parameter dependency, or error condition, use the detailed block specification.

The following specifications provide background for the interfaces and architecture described in this datasheet:

- **RISC-V Specifications:** <https://riscv.org/technical/specifications/>
- **AMBA AXI4-Lite Specification:** <https://developer.arm.com/documentation/ih0022/latest/>